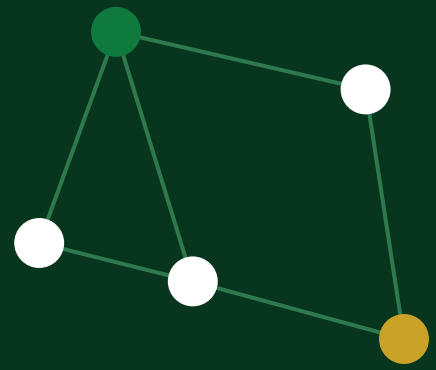
emsi

École Marocaine des Sciences de l'Ingénieur



CHAPITRE 2

Fondements de l'intelligence artificielle

Recherche, heuristiques, métaheuristiques, apprentissage automatique et réseaux de neurones

OBJECTIFS PÉDAGOGIQUES

À la fin de ce chapitre, vous saurez…

1. **Modéliser un problème**
   le représenter comme un espace d'états (états, actions, état initial, but).

2. **Dérouler BFS et DFS**
   simuler pas à pas les listes OPEN / CLOSED et comparer leurs propriétés.

3. **Appliquer GBFS et A***
   utiliser une heuristique, calculer f(n) = g(n) + h(n) et juger l'optimalité.

4. **Concevoir un algorithme génétique**
   sélection, croisement, mutation et critères d'arrêt sur le TSP.

5. **Distinguer ML et Deep Learning**
   supervisé / non supervisé, perceptron et réseaux de neurones.

PLAN

Les six sections du chapitre

1

Espace de recherche

États, actions, graphe et arbre

2

Recherche aveugle

BFS, DFS, coût uniforme

3

Recherche heuristique

Heuristiques, GBFS, A*

4

Métaheuristiques

Algorithme génétique, TSP

5

Machine Learning

Supervisé, non supervisé

6

Deep Learning

Neurone, perceptron, réseaux

Des travaux dirigés (TD) ponctuent les sections 2, 3 et 4.

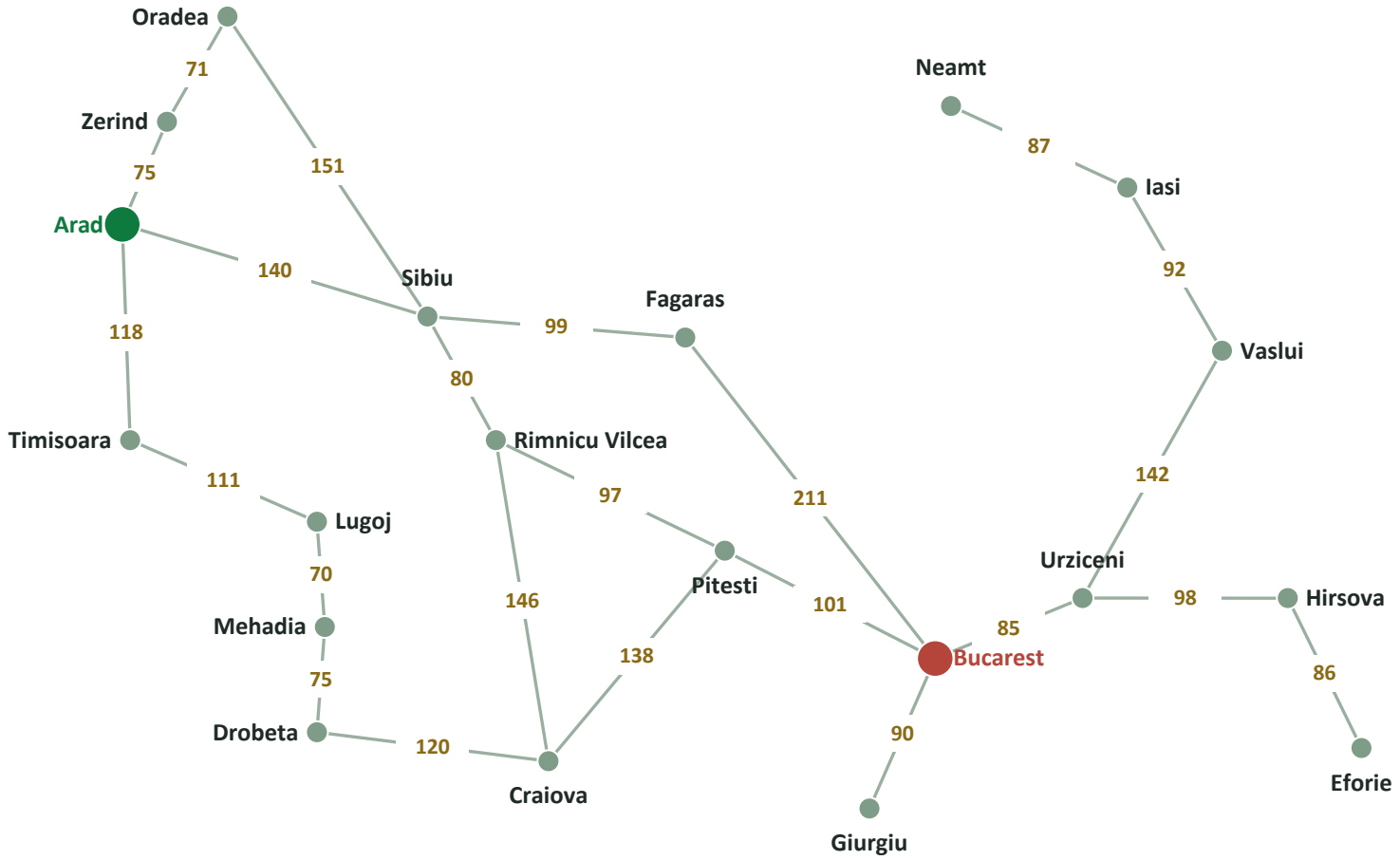
1

SECTION 1

L'espace de recherche

« Comment aller d'Arad à Bucarest ? »

Un problème concret : la carte de Roumanie



La situation

- **Départ :** Arad
- **Arrivée :** Bucarest
- 20 villes reliées par des routes
- Chaque route porte sa distance en km
- **Question :** quel itinéraire choisir ?

Qu'est-ce qu'un espace de recherche ?

Pour résoudre un problème avec l'IA, il faut d'abord le **représenter de manière formelle**. On le modélise par un **espace de recherche** (ou espace d'états).

Espace de recherche

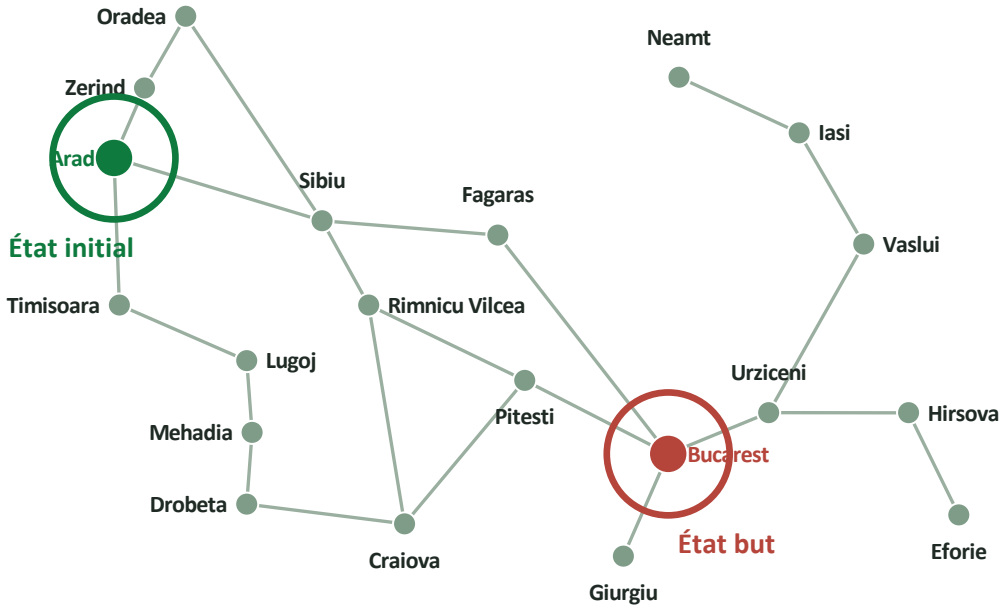
l'ensemble de tous les états possibles d'un problème et des chemins qui les relient.

État

une situation du problème (ex. « être à Sibiu »).

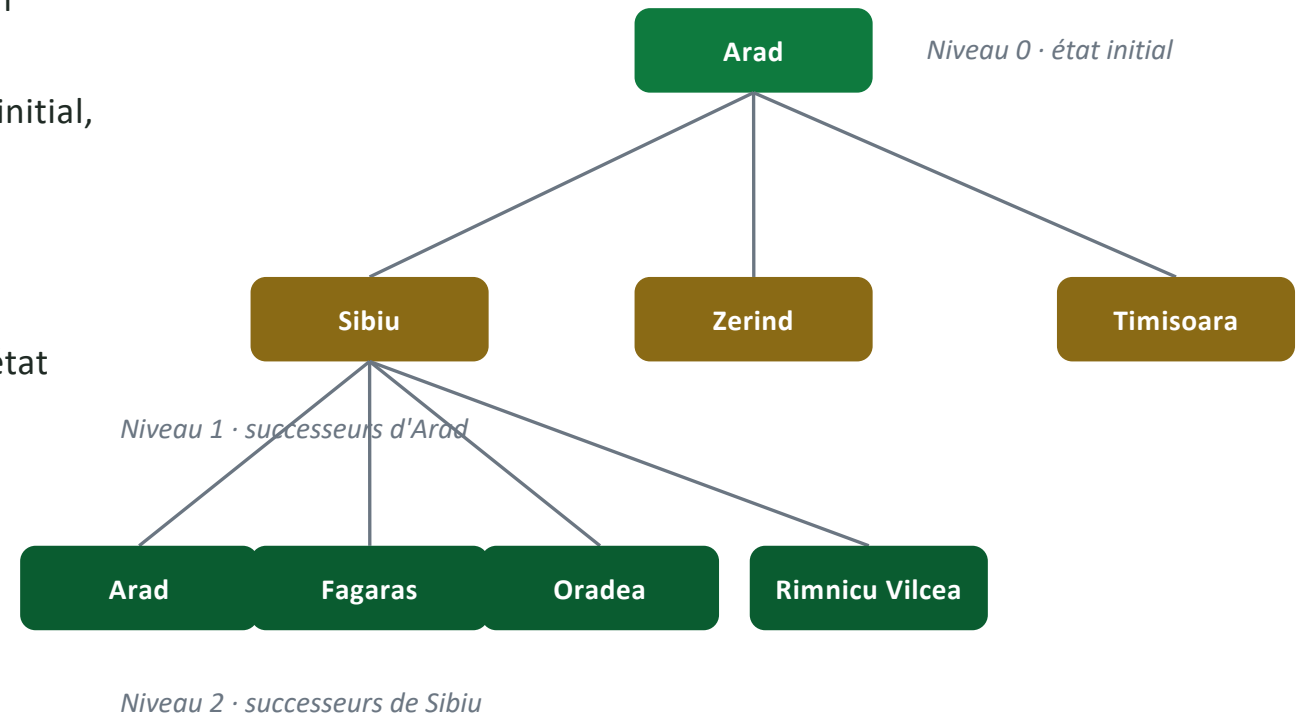
Action

permet de passer d'un état à un autre (ex. « prendre la route vers Fagaras »).



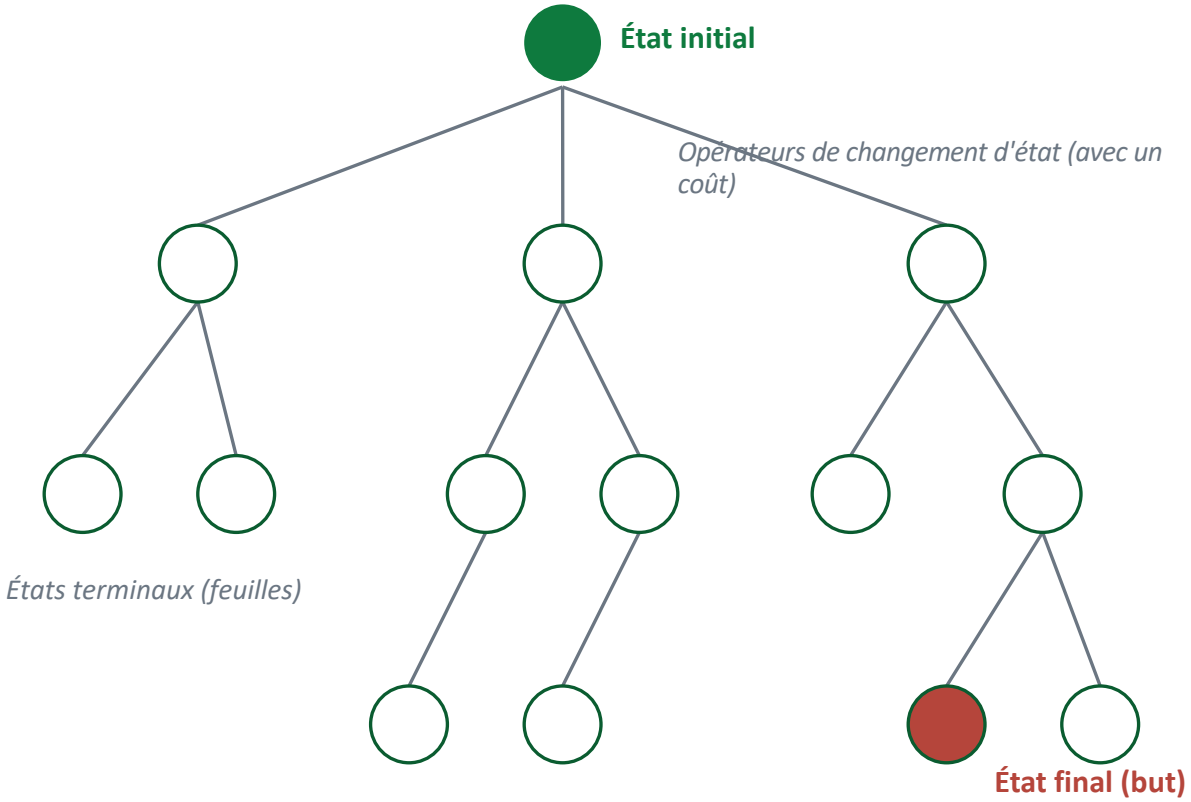
Du graphe à l'arbre de recherche

- L'ensemble des états et des actions se représente par un graphe ou un arbre de recherche.
- Traverser cet arbre = explorer les états à partir de l'état initial, en suivant les actions possibles.
- **Découvert :** état atteint pour la première fois.
- **Traité :** toutes ses actions ont été examinées.
- **But :** trouver un chemin solution reliant l'état initial à l'état but.



Remarque : Arad réapparaît comme successeur de Sibiu — d'où l'utilité de mémoriser les états déjà traités (CLOSED).

Les structures de la recherche : OPEN et CLOSED



OPEN

Frontière : nœuds découverts mais pas encore développés. Sa structure dépend de l'algorithme (file, pile, file de priorité).

CLOSED

Nœuds déjà développés (visités), mémorisés pour éviter les re-visites et les boucles.

SOLUTION

Un chemin : la séquence d'états qui mène de l'état initial à l'état final.

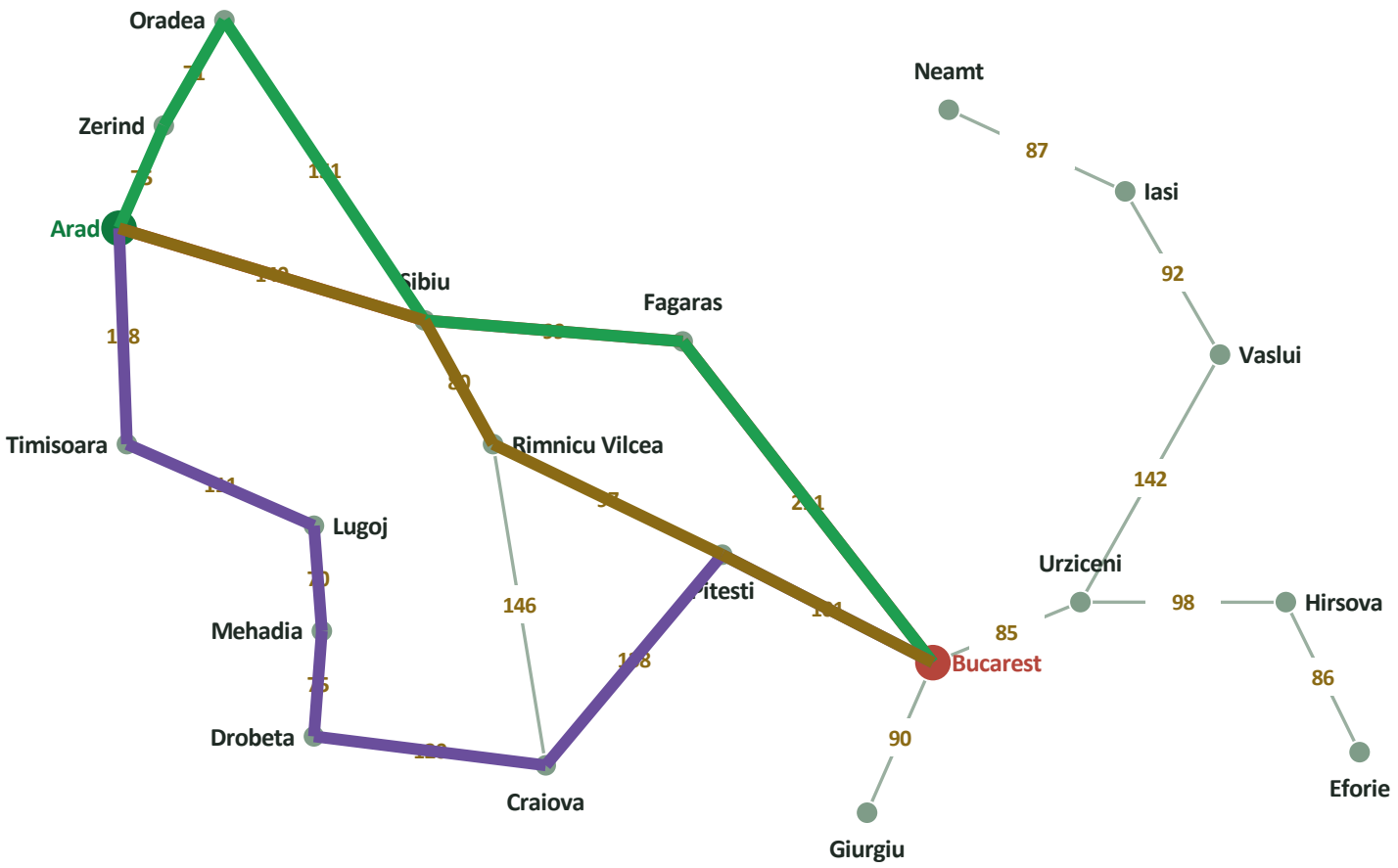
Le processus général de recherche

Initialiser : OPEN = {état initial}, CLOSED = ∅, parent[·] = ∅ puis répéter les étapes 1 à 5 :

1. **Choisir —** dans OPEN le nœud n selon une stratégie : le plus ancien, le plus récent, le plus « prometteur », le moins coûteux…
2. **Tester le but —** si n est l'état but → reconstruire le chemin via parent[·] et s'arrêter.
3. **Développer n —** appliquer les opérateurs pour générer ses successeurs.
4. **Enregistrer —** chaque successeur v non déjà vu : parent[v] = n, puis l'ajouter à OPEN.
5. **Déplacer —** n de OPEN vers CLOSED, puis recommencer.

Toute la différence entre BFS, DFS, GBFS et A tient à l'étape 1 : la stratégie de choix dans OPEN.*

Plusieurs chemins solutions possibles



Chemin 1

140 + 99 + 211 = 450 km

Chemin 2

75 + 71 + 151 + 99 + 211 = 607 km

Chemin 3

118 + 111 + 70 + 75 + 120 + 138 + 101 = 733 km

Chemin 4

140 + 80 + 97 + 101 = 418 km

Le plus court : 418 km. Comment un algorithme le trouve-t-il ?

QUIZ · À VOUS DE JOUER

Pour aller d'Arad à Bucarest, quel est l'état initial, l'état but, et que représente une action ?

- Initial : Arad
- But : Bucarest
- Action : emprunter une route
- Action : choisir une ville au hasard

Réponse État initial = Arad, état but = Bucarest ; une action = emprunter une route vers une ville voisine (avec un coût en km).

2

SECTION 2

Les algorithmes de recherche aveugle

« Explorer sans boussole »

Trois grandes familles d'algorithmes de recherche

1

Algorithmes aveugles

Explorent l'espace d'états sans information supplémentaire sur le problème.

BFS · DFS · coût uniforme

2

Algorithmes informés (heuristiques)

Utilisent une estimation h(n) de la distance au but pour guider l'exploration.

GBFS · A*

3

Algorithmes métaheuristiques

Font évoluer une population de solutions pour approcher l'optimum des problèmes trop vastes.

Algorithme génétique · recuit simulé…

Plus d'information sur le problème → exploration plus ciblée

Les algorithmes aveugles (non informés)

Ils explorent l'espace d'états sans information supplémentaire : seule la structure du graphe est connue.

Recherche en largeur (BFS)

Développe les nœuds niveau par niveau.

Structure : file FIFO

Recherche en profondeur (DFS)

Descend le plus loin possible, puis revient en arrière.

Structure : pile LIFO

Recherche à coût uniforme (UCS)

Développe d'abord le nœud de plus faible coût cumulé g(n).

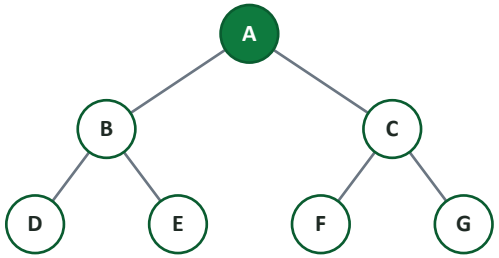
Structure : file de priorité

Le coût uniforme (proche de l'algorithme de Dijkstra) garantit le chemin de coût minimal ; sur la carte de Roumanie, il retrouve les 418 km.

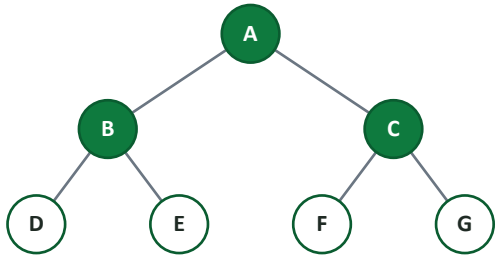
Principe : explorer niveau par niveau

La recherche en largeur (*Breadth-First Search*) développe les nœuds par niveau. On part de l'état initial, puis on ajoute à la frontière tous les états obtenus par les actions possibles. Ces états sont stockés dans une **file FIFO** (First In, First Out) : les premiers insérés sont les premiers explorés.

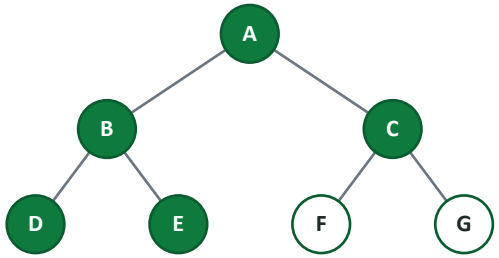
- On développe tous les nœuds du niveau i avant ceux du niveau i + 1.
- Les nouveaux successeurs sont placés à la fin de la file.



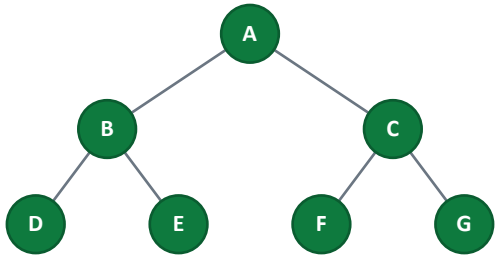
Niveau 0



Niveau 1



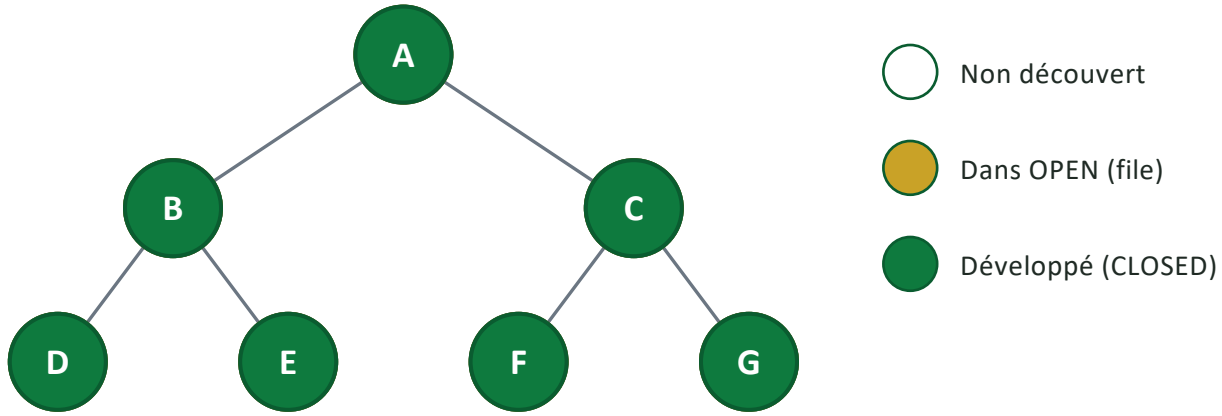
Niveau 2 (début)



Niveau 2

Ordre de visite : A – B – C – D – E – F – G

Déroulé pas à pas : OPEN est une file



Étape	Nœud développé	OPEN (file)	CLOSED (ordre de visite)
0	—	[A]	∅
1	**A**	[B, C]	A
2	**B**	[C, D, E]	A B
3	**C**	[D, E, F, G]	A B C
4	**D**	[E, F, G]	A B C D
5	**E**	[F, G]	A B C D E
6	**F**	[G]	A B C D E F
7	**G**	[ ]	A B C D E F G

```python
from collections import deque

def bfs(graph, start):
    # File (FIFO) des états à explorer
    queue = deque([start])
    # États déjà découverts
    visited = set([start])

    while queue:
        # Retirer le premier élément de la file
        node = queue.popleft()
        print("Visite :", node)

        # Parcourir les voisins (successeurs)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Avantages et inconvénients du BFS

✓ Avantages

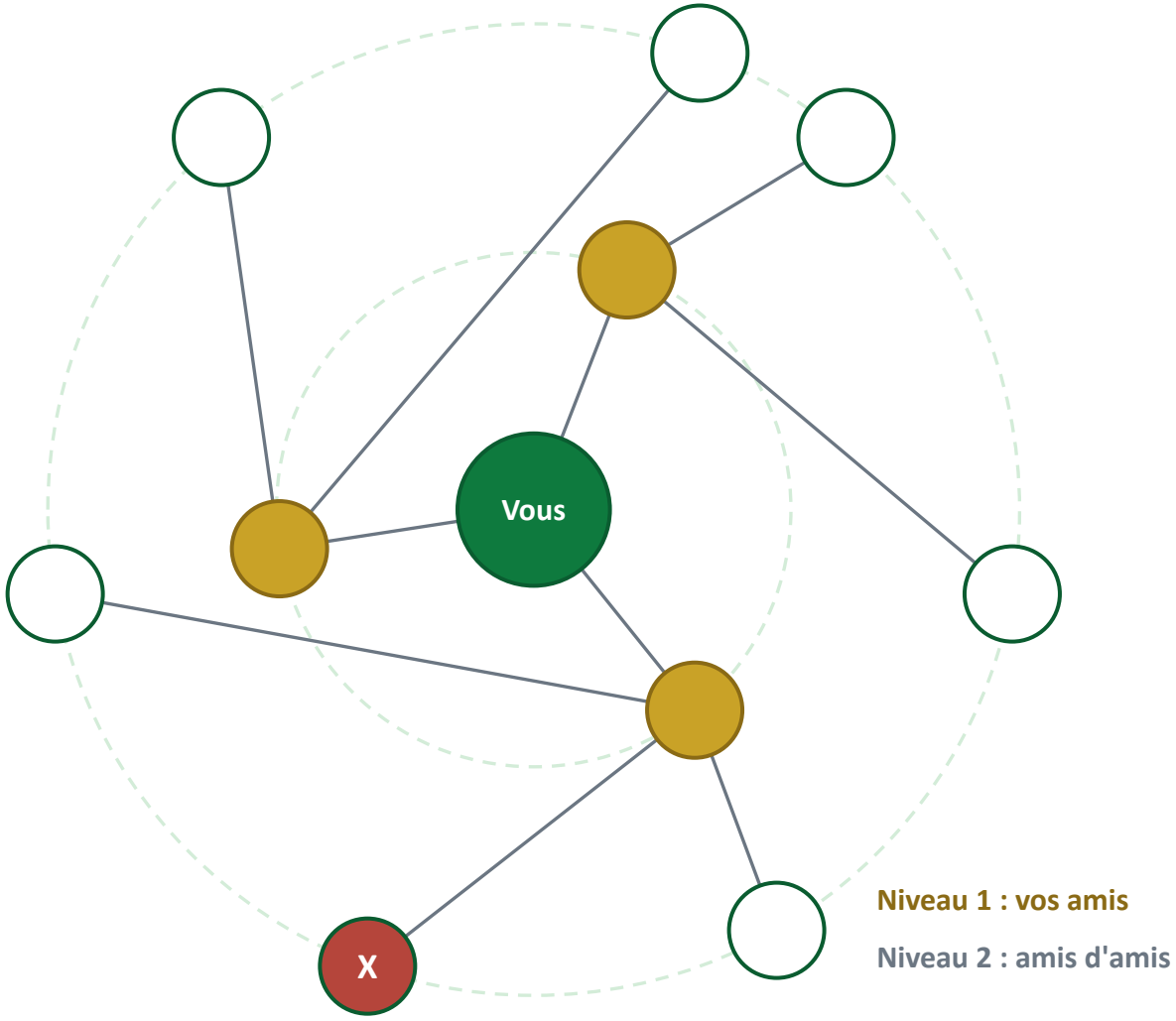
- **Trouve toujours la solution la plus courte** (en nombre d'actions).
- **Exploration systématique** (niveau par niveau).
- Facile à comprendre et à implémenter.
- **File FIFO** → ordre d'exploration garanti.

✗ Inconvénients

- **Mémoire importante :** il faut stocker tous les nœuds d'un niveau avant de passer au suivant.
- **Temps élevé** si l'espace de recherche est très grand.
- Peu adapté quand la solution est **profonde mais unique.**

Exemple : calculer la distance sociale

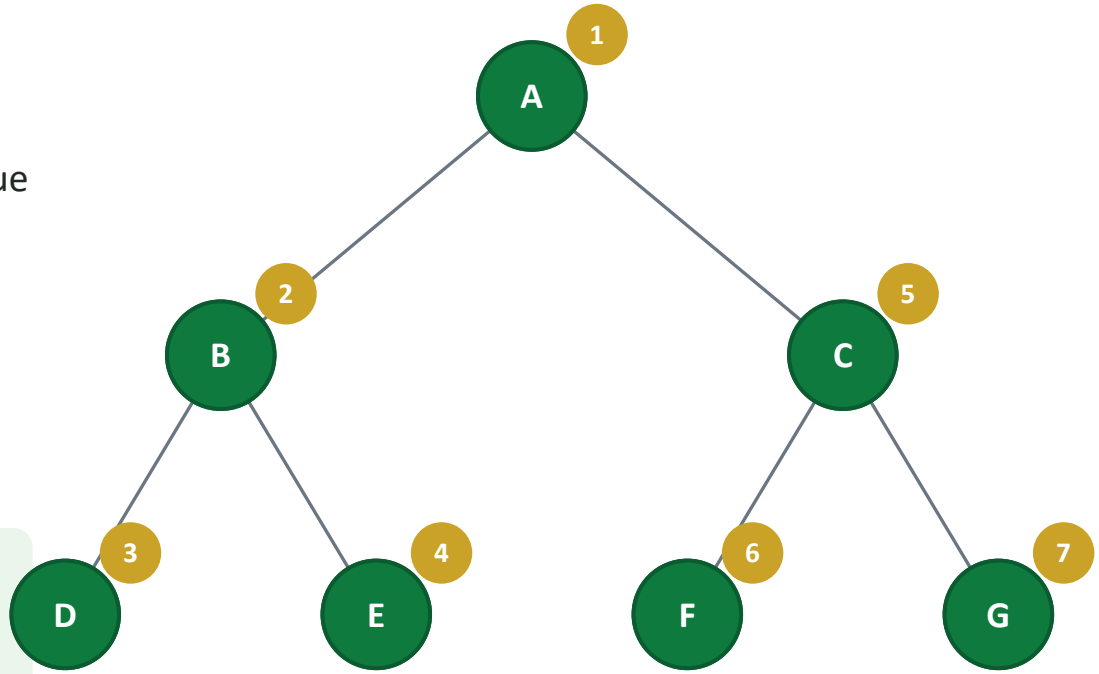
- Sur LinkedIn ou Facebook, on peut lire « vous êtes à 2 connexions de X ».
- Pour calculer ce **degré de séparation**, l'algorithme part de vous et explore **niveau par niveau** : vos amis, puis les amis de vos amis, etc.
- Le premier niveau où X apparaît donne la distance minimale : c'est exactement le BFS.



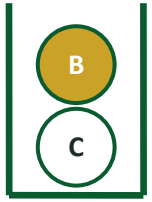
X est à 2 connexions de vous.

Principe : descendre le plus loin possible

La recherche en profondeur (*Depth-First Search*) descend le plus loin possible dans l'arbre avant de revenir en arrière. On choisit un successeur et l'on continue jusqu'à un nœud sans successeur ou déjà visité ; l'algorithme effectue alors un **retour arrière (backtracking)** pour explorer d'autres branches.



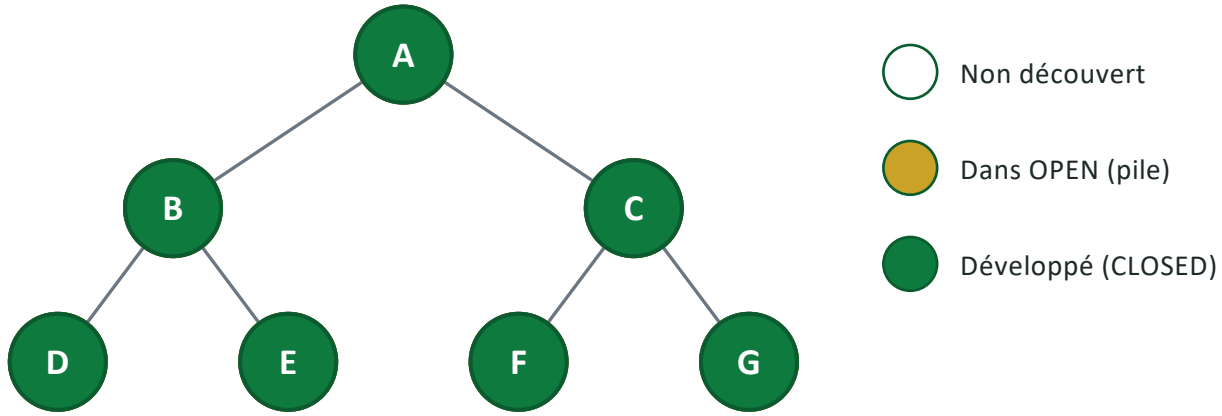
Pile LIFO (Last In, First Out) : le dernier état inséré est le premier exploré. Les nouveaux états vont au sommet de la pile.



← *sommet : sorti en premier*

Ordre de visite : A – B – D – E – C – F – G

Déroulé pas à pas : OPEN est une pile



Étape	Nœud développé	OPEN (pile)	CLOSED (ordre de visite)
0	—	[A]	∅
1	**A**	[C, B]	A
2	**B**	[C, E, D]	A B
3	**D**	[C, E]	A B D
4	**E**	[C]	A B D E
5	**C**	[G, F]	A B D E C
6	**F**	[G]	A B D E C F
7	**G**	[ ]	A B D E C F G

```python
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()

    # Marquer le nœud comme visité
    visited.add(start)
    print("Visite :", start)

    # Explorer les voisins un par un (en profondeur)
    for neighbor in graph[start]:
        if neighbor not in visited:
            print(f"-> {start} vers {neighbor}")
            dfs(graph, neighbor, visited)
            # Backtracking (retour arrière)
            print(f"<- retour à {start}")
```

Avantages et inconvénients du DFS

✓ Avantages

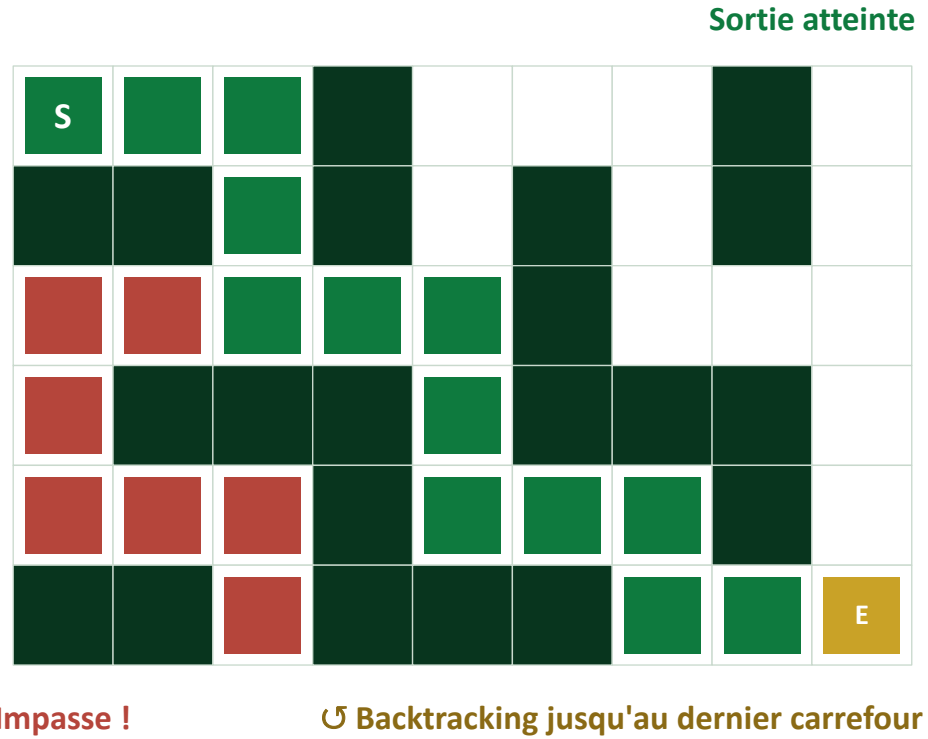
- **Peu de mémoire :** seule la pile des nœuds en cours est conservée.
- **Exploration rapide en profondeur :** atteint vite un état profond.
- **Implémentation simple** avec la récursion.
- **Utile pour un parcours complet** (ex. détection de cycles).

✗ Inconvénients

- **Ne garantit pas la solution la plus courte** (contrairement au BFS).
- Peut s'enfoncer dans une branche inutile → **perte de temps.**
- **Risque de boucle infinie** si le graphe a des cycles et qu'on ne mémorise pas les visités.
- Moins adapté quand la solution est proche de la racine.

Exemple : un robot explore un labyrinthe

- Un robot doit trouver la sortie d'un grand labyrinthe.
- Le DFS choisit un chemin et avance en profondeur jusqu'à la sortie… ou une impasse.
- Si le chemin est bloqué : **backtracking** (retour arrière) pour tester une autre possibilité.
- **Efficace :** il explore tout le labyrinthe avec peu de mémoire, mais ne garantit pas le chemin le plus court.



SECTION 2 · SYNTHÈSE

BFS ou DFS : comment choisir ?

Critère	BFS (largeur)	DFS (profondeur)
Structure de OPEN	File FIFO	Pile LIFO (ou récursion)
Ordre d'exploration	Niveau par niveau	Branche par branche
Solution la plus courte (en actions)	Oui (garanti)	Non
Mémoire	Élevée : O(b^d)	Faible : O(b·m)
Risque	Explosion mémoire	Boucles, branches inutiles
Usage typique	Plus court chemin, réseaux sociaux	Labyrinthes, parcours complet, cycles

b = facteur de branchement · d = profondeur de la solution · m = profondeur maximale de l'arbre

QUIZ · FILE OU PILE ?

Quel algorithme utilise une pile (LIFO) et ne garantit pas le chemin le plus court ?

- BFS
- **DFS**
- Coût uniforme

Réponse DFS : il dépile toujours le dernier nœud ajouté et peut trouver un chemin plus long que le BFS.

TD

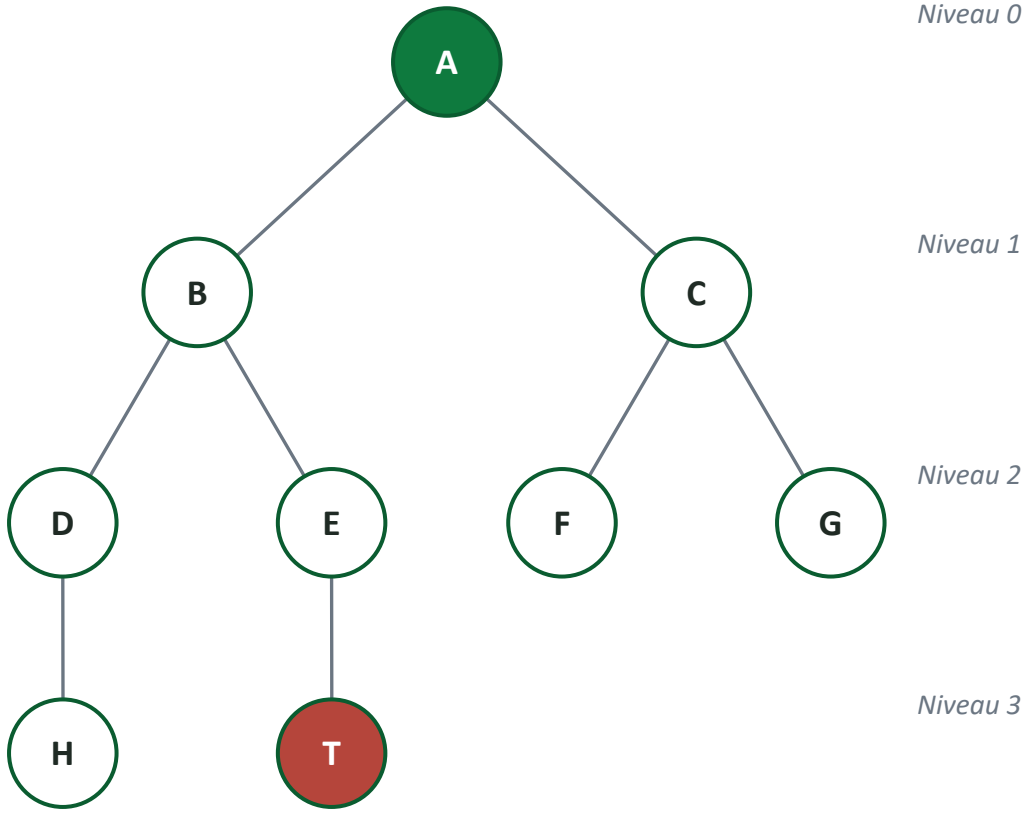
TRAVAUX DIRIGÉS 1

Recherche aveugle : BFS et DFS

Simuler, comparer, justifier

Trouver la cible T dans un arbre

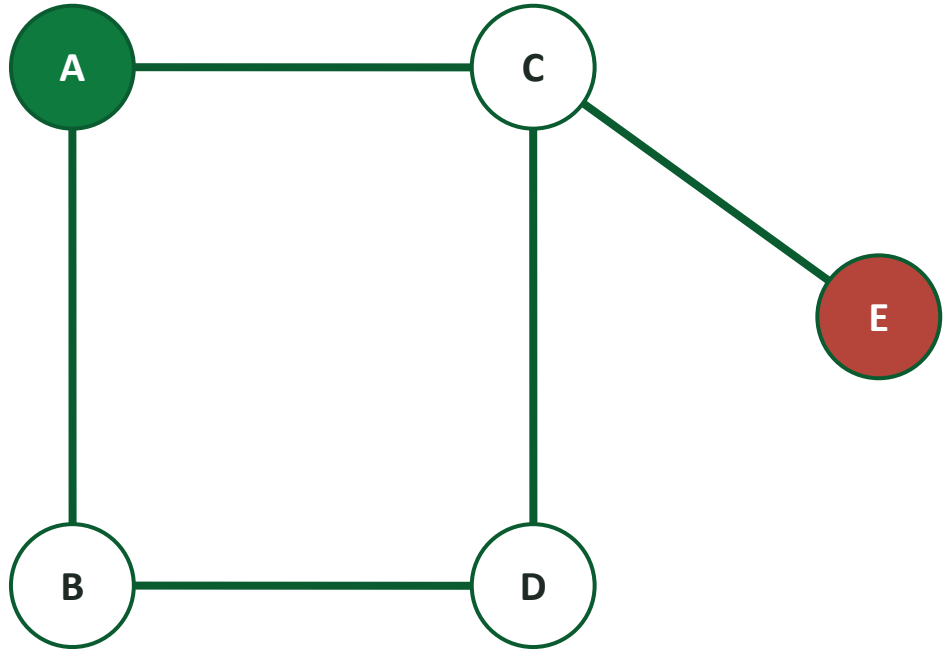
- A est la racine ; A a deux enfants : B (à gauche) et C (à droite).
- B a deux enfants : D et E ; C a deux enfants : F et G.
- D a un enfant : H ; E a un enfant : T (la cible). F et G n'ont pas d'enfants.
- **Règle de visite :** toujours explorer de la gauche vers la droite.



- **BFS :** ordre des nœuds visités ? chemin le plus court (en liens) de A à T ?
- **DFS :** ordre des nœuds visités ? chemin trouvé ?
- **Analyse :** pourquoi le BFS garantit-il l'optimalité, et pourquoi le DFS peut-il aboutir à un chemin plus long ?

Gestion du trafic : du carrefour A au carrefour E

- Chaque lettre est un carrefour, chaque ligne une route.
- **Départ :** A · **Cible :** E
- **Règle :** explorer les voisins par ordre alphabétique (B avant C, D avant E…).
- Routes : A–B, A–C, B–D, C–D, C–E.

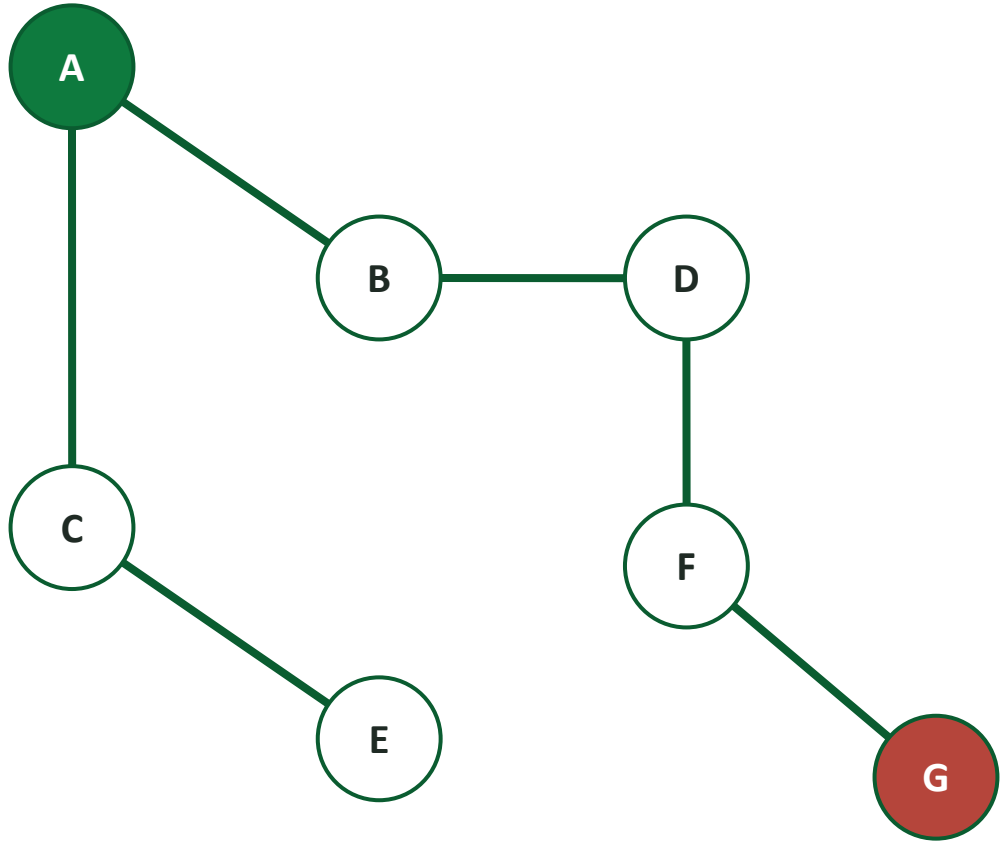


- **BFS :** séquence des carrefours visités jusqu'à E ? nombre minimal de changements de route de A à E ?
- **DFS :** séquence visitée jusqu'à E ? chemin trouvé ?
- **Analyse :** pourquoi un service comme Google Maps privilégie-t-il le BFS (ou ses variantes pondérées) ? Dans quel cas utiliser le DFS ici ?

Du carrefour A au carrefour G

- **Départ :** A · **Cible :** G · ordre alphabétique
- Routes : A–B, A–C, B–D, C–E, D–F, F–G.
- E est une impasse (feuille).

- Simulez le BFS : listez les carrefours visités et donnez le chemin trouvé.
- Simulez le DFS : même question.
- Comparez le nombre de nœuds visités par chaque algorithme. Lequel est le plus économe ici, et pourquoi ?



3

SECTION 3

La recherche heuristique (informée)

« Et si l'on avait une boussole ? »

Recherche heuristique : définition générale

La recherche heuristique regroupe des méthodes qui explorent un espace d'états en donnant **priorité aux pistes qui semblent rapprocher le plus du but**. Elle s'appuie sur une estimation ***h(n)***, appelée heuristique, du coût (ou de la distance) restant depuis l'état n jusqu'à l'objectif, afin de choisir quel nœud étendre en premier — et ainsi réduire le nombre d'états visités et le temps de calcul.

◎ **But :** trouver une solution plus vite en guidant l'exploration.

→ **Principe :** priorité aux nœuds qui semblent les plus proches du but selon h(n).

GBFS

Greedy Best-First Search

choisit le plus petit h(n)

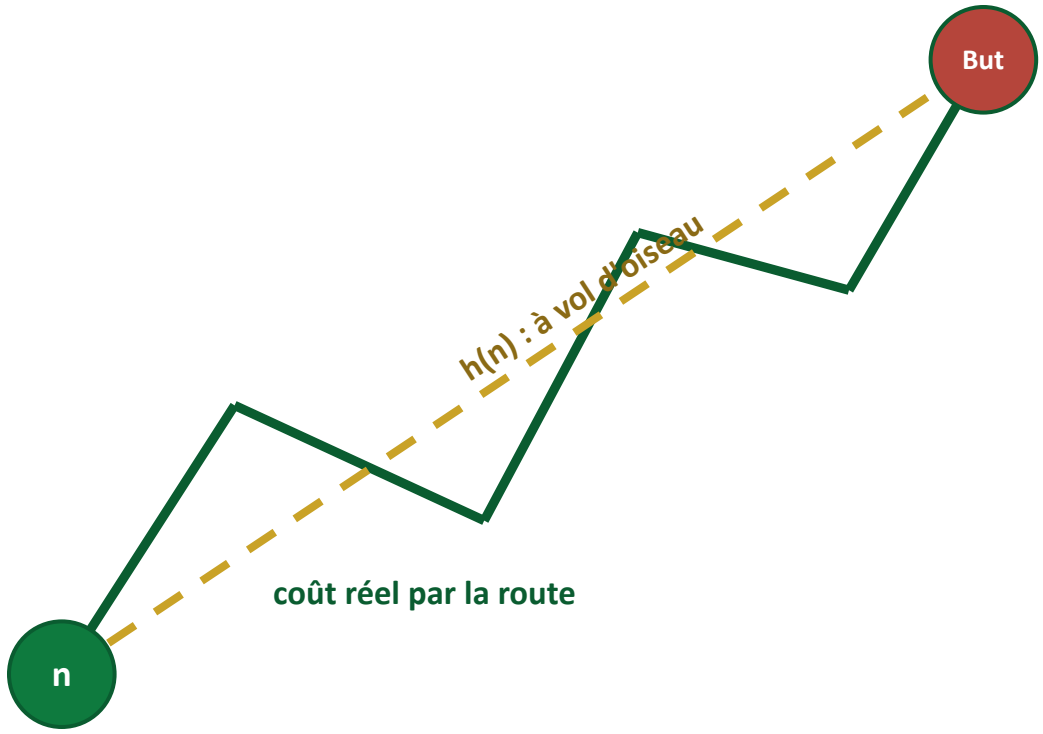
A*

« A étoile »

choisit le plus petit f(n) = g(n) + h(n)

Qu'est-ce qu'une heuristique ?

- Une heuristique est une **estimation rapide** (de la distance pour atteindre l'état but, par exemple.)
- Elle **oriente** la recherche là où cela semble le plus prometteur, pour **éviter d'explorer partout.**
- Elle n'est pas exacte : c'est une « boussole », pas une carte complète.



Exemple (carte de Roumanie) : h(n) = distance à vol d'oiseau entre la ville n et Bucarest. Elle se calcule immédiatement et ne dépasse jamais la distance réelle par la route.

ultra-classiques

Distance de Manhattan (L1)

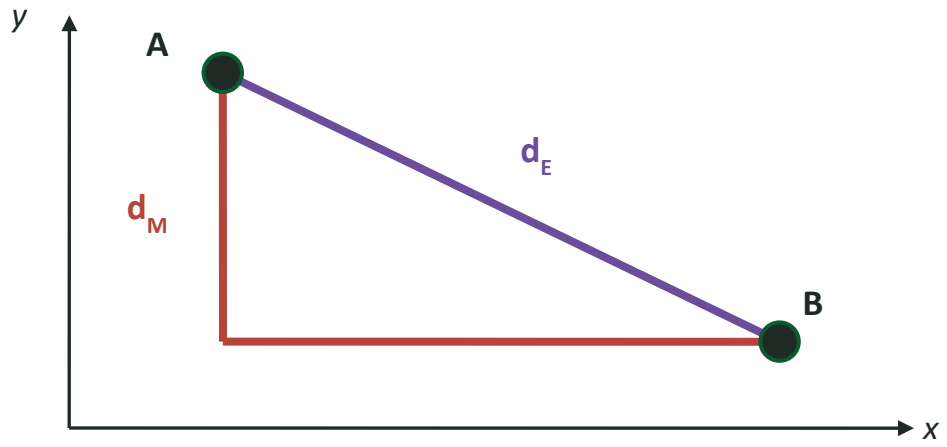
$$d_M = |x_A - x_B| + |y_A - y_B|$$

Distance parcourue en suivant un chemin « en damier », comme dans les rues d'une ville quadrillée.

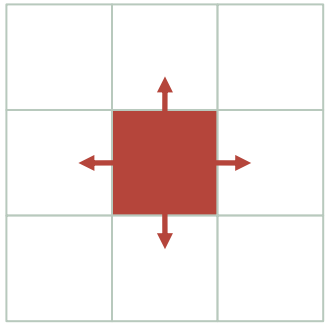
Distance euclidienne (L2)

$$d_E = \sqrt{((x_B - x_A)^2 + (y_B - y_A)^2)}$$

Distance en ligne droite (« à vol d'oiseau ») entre $A(x_A, y_A)$ et $B(x_B, y_B)$.

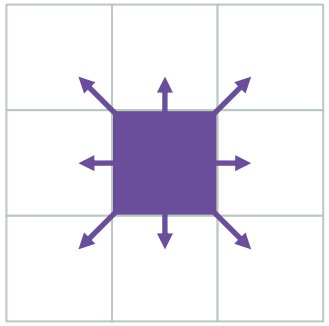


Quand utiliser quelle distance ?



Manhattan (L1)

Déplacements N / E / S / O uniquement : grille à 4 voisins, rues en damier.



Euclidienne (L2)

Diagonales permises ou espace continu : grille à 8 voisins, « à vol d'oiseau ».

Exemple chiffré

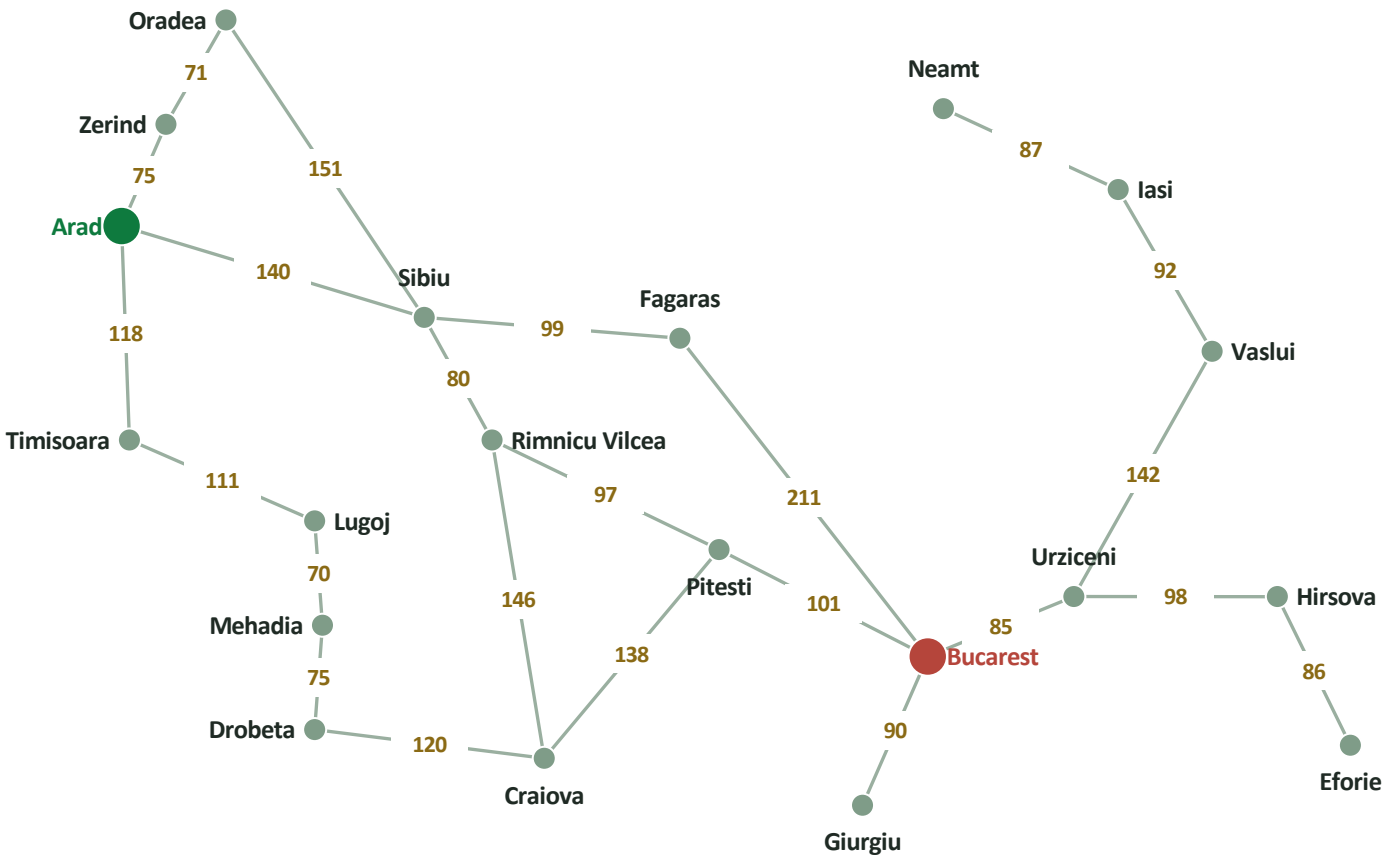
A = (2, 5) B = (7, 2)

$$d_M = |2 - 7| + |5 - 2| = 5 + 3 = \mathbf{8}$$

$$d_E = \sqrt{(5^2 + 3^2)} = \sqrt{34} \approx \mathbf{5{,}83}$$

On a toujours $d_E \leq d_M$: la ligne droite est le plus court chemin.

La carte et l'heuristique h(n) vers Bucarest



Distance à vol d'oiseau jusqu'à Bucarest

Ville	h	Ville	h
Arad	366	Mehadia	241
Bucarest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

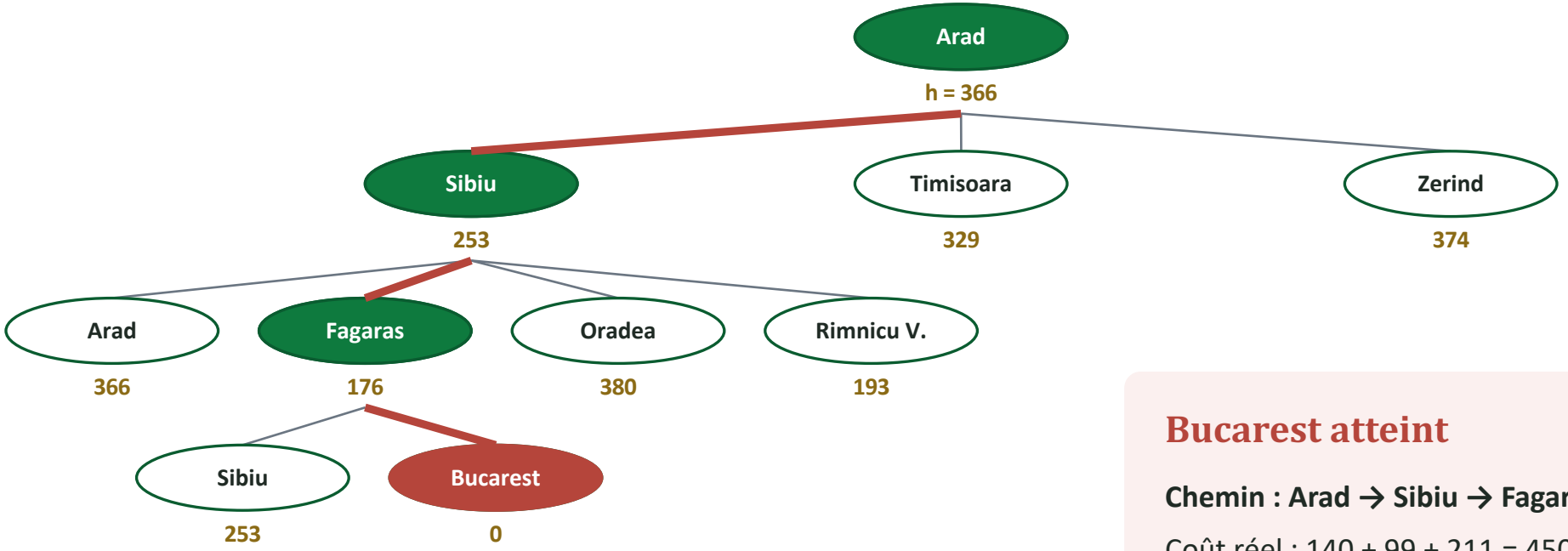
GBFS : la recherche gloutonne guidée par h(n)

Méthode qui explore un espace d'états en donnant **priorité au nœud dont l'heuristique h(n) est la plus petite**, c'est-à-dire celui qui semble le plus proche du but. À chaque étape, on extrait ce nœud de OPEN (**file de priorité triée par h**), on étend ses voisins, on les insère avec leur valeur h(·), puis on recommence.

« Glouton » : il ne regarde que ce qu'il reste à parcourir, jamais le chemin déjà payé.

```
1. Initialiser
   OPEN   ← { s }  avec la clé h(s)
   CLOSED ← ∅ ;  parent[s] ← ∅
2. Tant que OPEN n'est pas vide :
   a. n ← extraire de OPEN le nœud de plus petit h(n)
   b. Si n = but : retourner le chemin (via parent)
   c. Si n ∈ CLOSED : passer au suivant
   d. Ajouter n à CLOSED
   e. Pour chaque voisin v de n :
        si v ∉ CLOSED :
           parent[v] ← n
           insérer v dans OPEN avec la clé h(v)
3. OPEN vide : pas de solution trouvée
```

Exemple GBFS : d'Arad à Bucarest



Bucarest atteint

Chemin : Arad → Sibiu → Fagaras → Bucarest

Coût réel : 140 + 99 + 211 = 450 km

Rapide (3 développements), mais non optimal : le meilleur chemin fait 418 km.

A* : choisir le nœud qui minimise f(n)

$$f(n) = g(n) + h(n)$$

g(n)

Coût cumulé déjà parcouru depuis le départ jusqu'à n

(ce que l'on a déjà payé)

h(n)

Coût restant estimé de n jusqu'au but

(ce qu'il resterait à payer)

f(n)

Coût total estimé d'une solution passant par n

(le critère de choix dans OPEN)

A combine le meilleur des deux mondes : la prudence du coût uniforme (g) et le flair du glouton (h).*

Principe et algorithme de A*

Principe

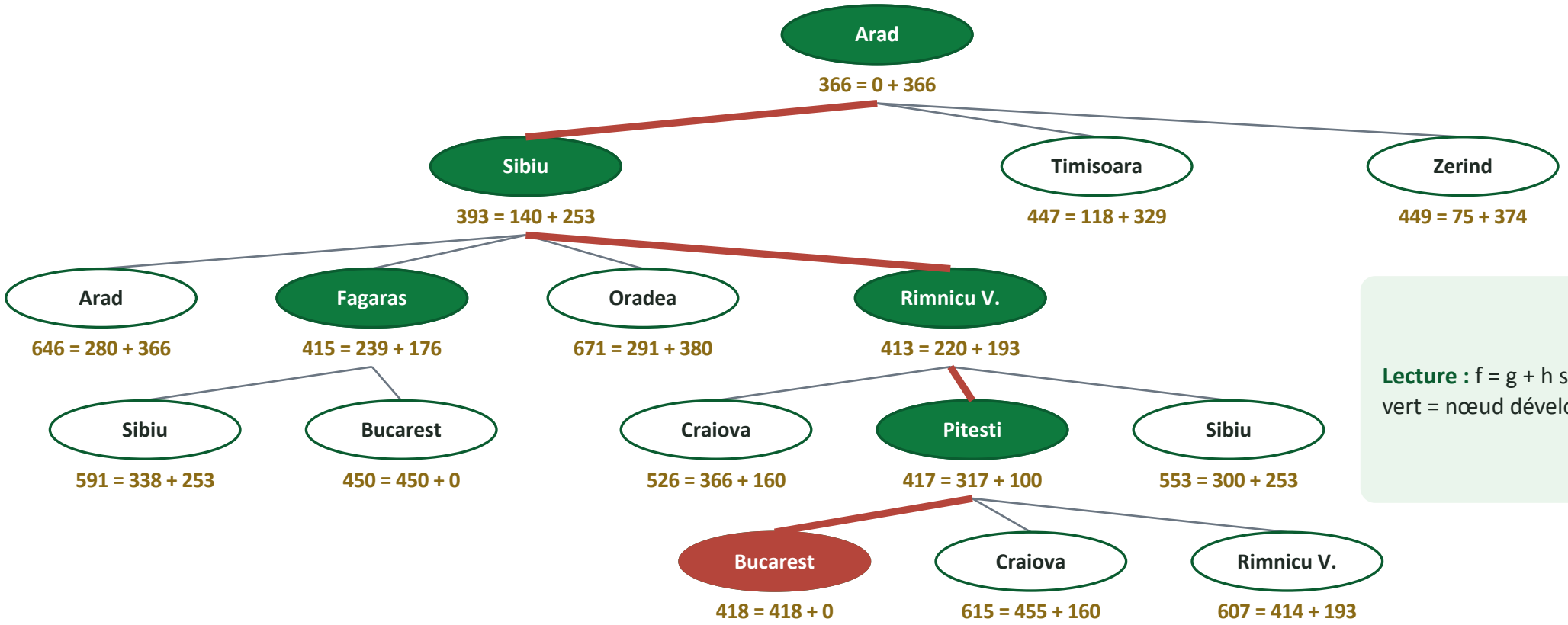
La liste OPEN est ordonnée selon la valeur **minimale de f**.

Si l'on atteint un nœud par un chemin de coût c, puis que l'on redécouvre ce même nœud par un chemin de coût **inférieur à c**, on élimine le premier chemin et l'on ne conserve que le second.

Exemple : Bucarest entre dans OPEN avec f = 450 (via Fagaras), puis est retrouvé avec f = 418 (via Pitesti) : seul le second chemin est gardé.

```
X0 ← nœud_initial
Open ← {X0} ; Closed ← ∅ ; g(X0) ← 0 ; X ← X0
Tant que Open ≠ ∅ faire
   Open ← Open − X ;  Closed ← Closed + X
   Si X = But : retourner la solution, stop
   Pour chaque Y ∈ Succ(X) faire
      Si Y ∉ (Open ∪ Closed)
         ou g(Y) > g(X) + coût(X, Y) alors
         g(Y) ← g(X) + coût(X, Y)
         f(Y) ← g(Y) + h(Y)
         parent(Y) ← X
         Open ← Open + Y  (ordre croissant de f)
   Si Open ≠ ∅ alors X ← tête(Open)
Si Open = ∅ : le problème n'a pas de solution
```

Exemple A* : d'Arad à Bucarest



Lecture : f = g + h sous chaque ville ; vert = nœud développé.

Arad → Sibiu → Rimnicu V. → Pitesti → Bucarest = 418 km (optimal)

Propriétés de A* : admissibilité et consistance

Admissible si h ne surestime jamais le coût réel h*(n) pour atteindre le but :

$$h(n) \leq h^*(n)$$

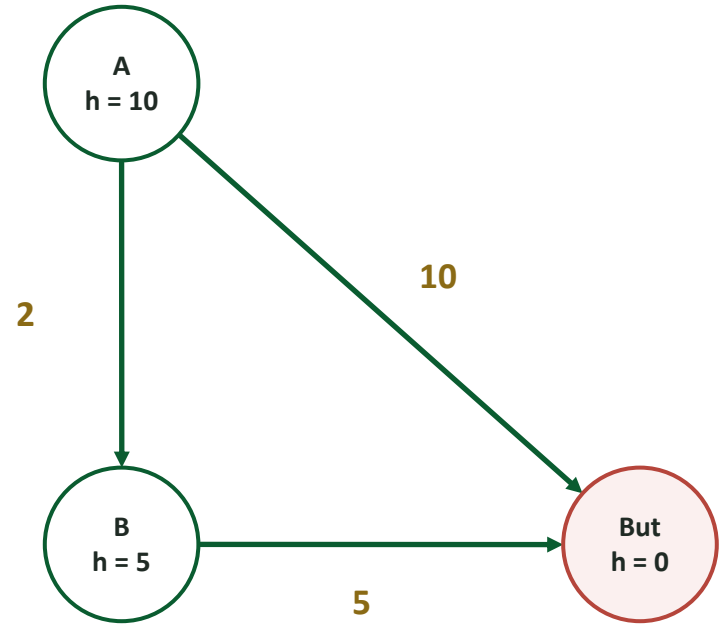
→ *A* trouve alors une solution optimale (en recherche arborescente).*

Consistante (monotone) si pour tout nœud n et tout successeur n' :

$$h(n) \leq \text{coût}(n, n') + h(n')$$

→ *A* reste optimal même en recherche dans un graphe.*

Toute heuristique consistante est admissible (la réciproque est fausse).



h(A) = 10 > coût(A,B) + h(B) = 2 + 5 = 7
⇒ h n'est pas consistante. Le vrai coût depuis A vaut 7 : h(A) le surestime, elle n'est donc pas admissible non plus.

GBFS ou A* : le match sur la carte de Roumanie

Critère	GBFS (glouton)	A*
Fonction d'évaluation	f(n) = h(n)	f(n) = g(n) + h(n)
Prend en compte le coût déjà parcouru	Non	Oui
Optimalité	Non garantie	Oui, si h est admissible
Résultat Arad → Bucarest	450 km (3 développements)	418 km (5 développements)
Usage typique	Réponse rapide « assez bonne »	GPS, jeux vidéo, robotique, logistique

QUIZ · CALCUL ÉCLAIR

À Fagaras, on a parcouru g = 239 km et h(Fagaras) = 176. Que vaut f, et quel algorithme l'utilise ?

- f = 239
- f = 176
- **f = 415 (A*)**
- f = 63

Réponse f = g + h = 239 + 176 = 415. GBFS, lui, n'utiliserait que h = 176.

TD

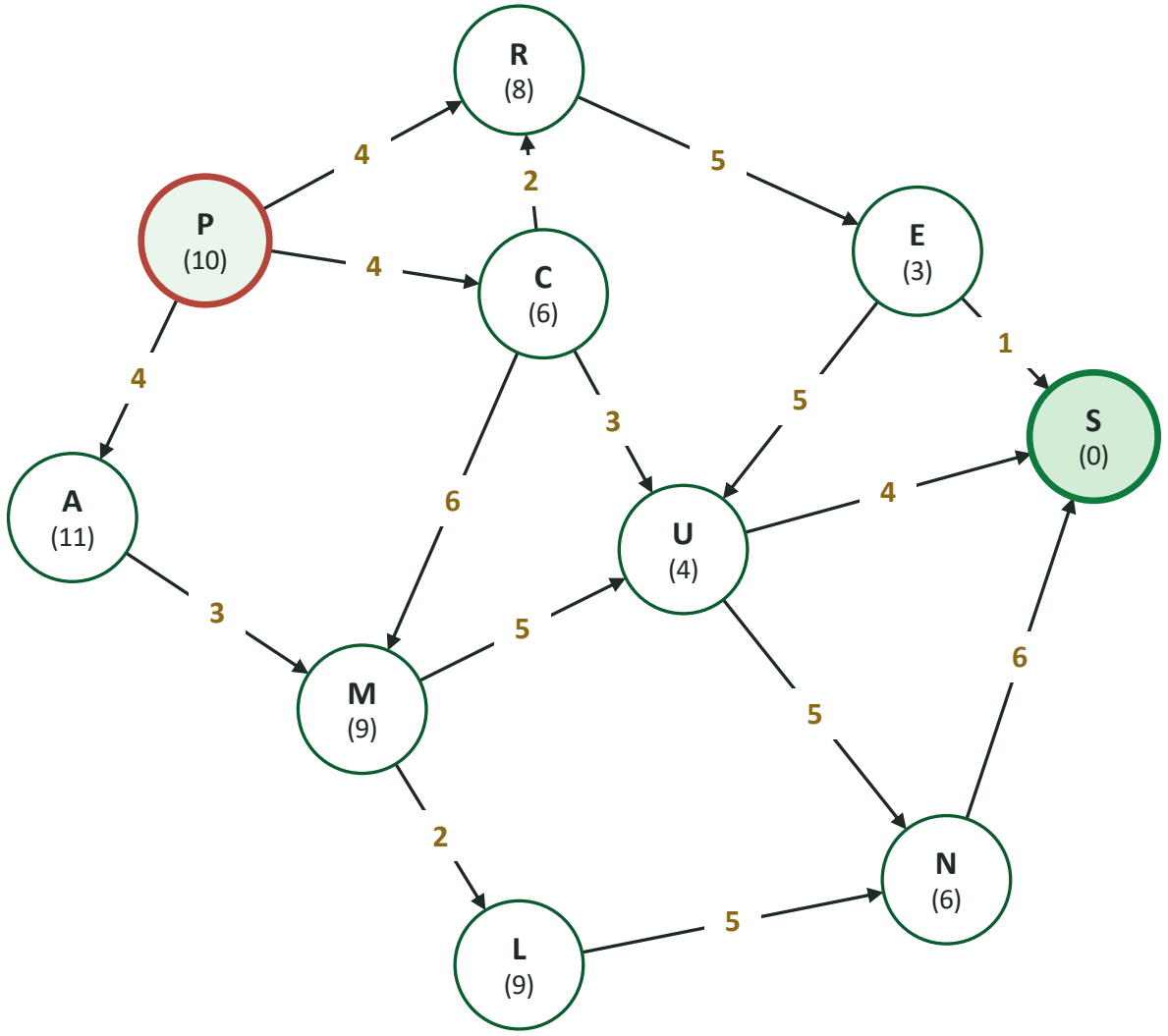
TRAVAUX DIRIGÉS 2

Recherche heuristique : GBFS et A*

Choisir le meilleur chemin, pas seulement le plus prometteur

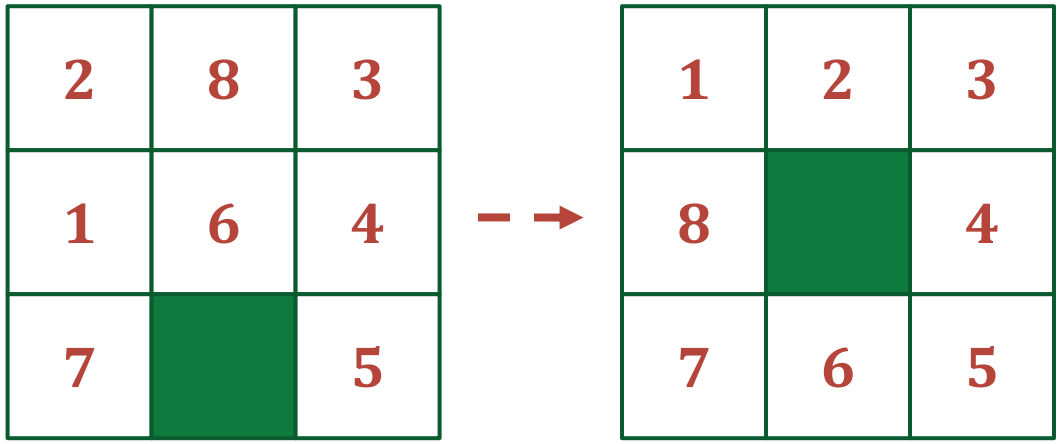
Meilleur chemin de P à S

- Comment un agent intelligent peut-il trouver le meilleur chemin pour aller de P à S ?
- Que représentent les valeurs h(n) indiquées dans chaque ville ?
- Que représente le coût réel c porté par chaque arc ?
- Trouvez le chemin en appliquant **GBFS**, puis **A***. Comparez.
- Bonus : l'heuristique est-elle admissible ? Justifiez.



Le jeu de taquin (8-puzzle)

- **État :** une configuration légale du jeu.
- **État initial :** la configuration de départ.
- **État final (but) :** la configuration gagnante.
- **h(n) :** nombre de cases mal placées par rapport à l'état final.
- **g :** coût égal à 1 pour chaque déplacement.

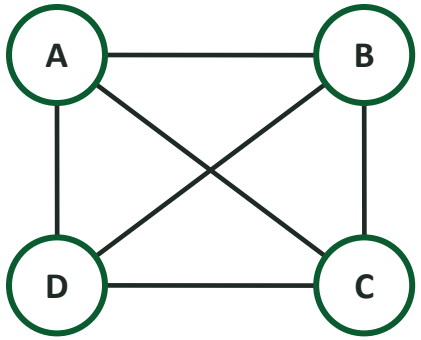


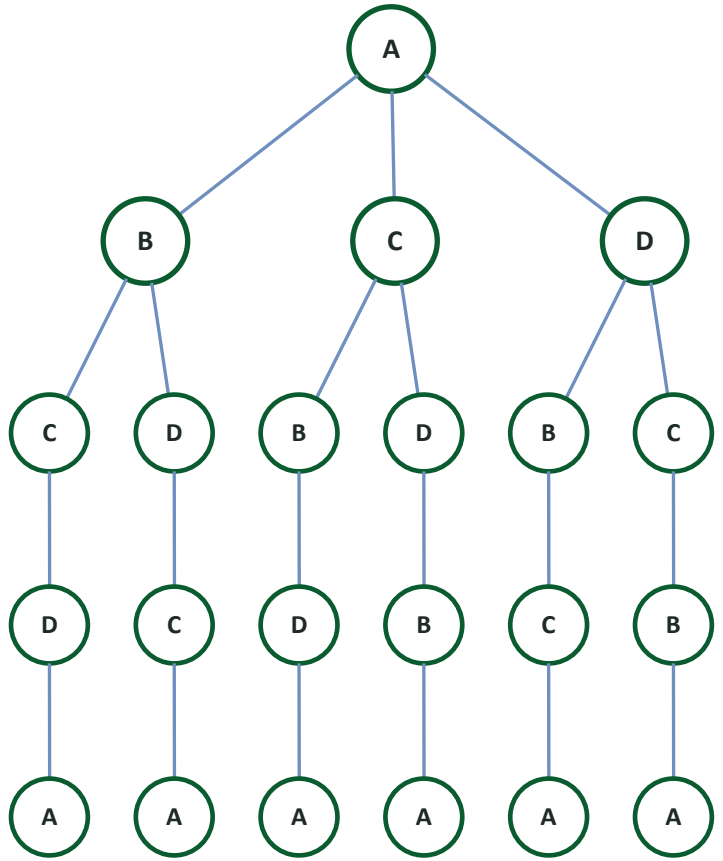
État initial — État final (but)

Questions : calculez h(état initial) ; développez l'arbre de recherche A* (f = g + h) jusqu'à l'état final ; donnez la suite de déplacements.

Le problème du voyageur de commerce (TSP)

Principe : le voyageur doit visiter toutes les villes **une seule fois** et revenir au point de départ, en cherchant le **chemin de coût minimal.**





6 feuilles = 3 circuits × 2 sens

Nombre de circuits pour n villes : $(n - 1)! / 2$ (on divise par 2 car un circuit et son inverse sont identiques).

Pour 4 villes : (4 – 1)! / 2 = 3 circuits à étudier.

Exemples en entreprise : tournées de livraison, collecte, maintenance, perçage de cartes électroniques.

A* face au TSP : l'explosion combinatoire

A* explore les chemins possibles en stockant pour chaque état le coût réel g(n), l'estimation h(n) et la liste des villes visitées.

4 villes

3

circuits

calcul instantané

10 villes

181 440

circuits

déjà lourd en mémoire

15 villes

43,6 milliards

≈ 12 h à 10^6 états/s

mémoire saturée (plusieurs Go)

20 villes

$6{,}1 \times 10^{16}$

≈ 1 900 ans à 10^6 états/s

hors de portée, même d'un supercalculateur

Explosion combinatoire : la taille de l'espace croît comme (n − 1)!. Il faut d'autres méthodes → les métaheuristiques.

SECTION 4

Les métaheuristiques

« Quand chercher partout devient impossible »

Qu'est-ce qu'une métaheuristique ?

Définition. Une métaheuristique est une stratégie algorithmique générale conçue pour guider d'autres heuristiques afin de trouver des **solutions approchées** à des problèmes d'optimisation difficiles, pour lesquels on ne connaît pas de méthode classique plus efficace.

AG

Algorithmes génétiques

inspirés de l'évolution darwinienne

RS

Recuit simulé

inspiré du refroidissement des métaux

CF

Colonies de fourmis

inspirées des phéromones des fourmis

EP

Essaims particulaires

inspirés des vols d'oiseaux

En entreprise : optimisation de tournées (logistique), plannings du personnel, ordonnancement de production, conception de réseaux.

L'algorithme génétique : une idée darwinienne

- Un algorithme génétique (AG) est une **métaheuristique inspirée de la théorie de l'évolution** (Holland, 1975).
- Il fait évoluer une population de solutions pour approcher la meilleure.
- Au lieu d'explorer un **arbre de recherche** comme BFS ou A*, l'AG travaille sur **plusieurs solutions à la fois**, appelées population.

Analogie : les individus les mieux adaptés (fitness élevée) ont plus de chances de se reproduire et de transmettre leurs gènes.

1

Sélection

garder les meilleures solutions.

2

Croisement

combiner deux solutions pour en créer de nouvelles.

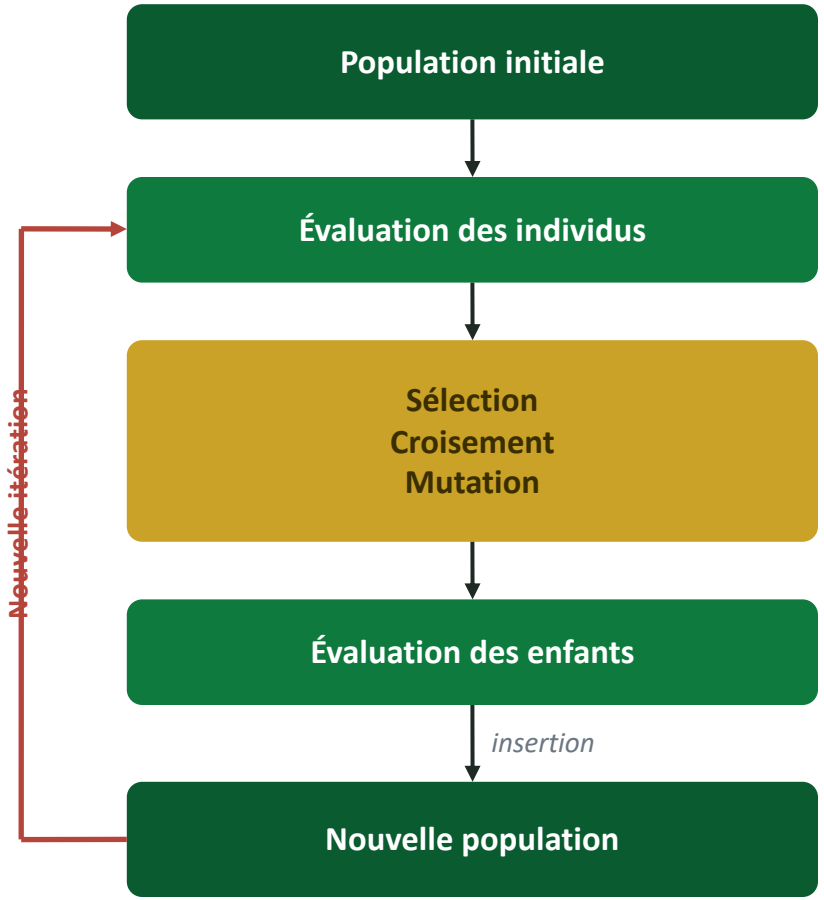
3

Mutation

introduire des changements aléatoires pour garder la diversité.

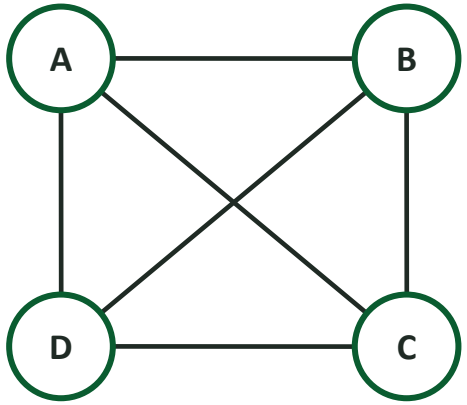
Vocabulaire et cycle de l'algorithme

- **Population :** ensemble de solutions candidates (chromosomes).
- **Fitness :** fonction d'évaluation de la qualité d'un individu.
- **Sélection :** choix des meilleurs individus (roulette, tournoi…).
- **Croisement (crossover) :** combinaison de deux parents pour créer des enfants.
- **Mutation :** modification aléatoire d'un gène pour introduire de la diversité.



La population initiale (exemple du TSP à 4 villes)

La **population** est un ensemble de solutions candidates générées aléatoirement. Chaque solution, appelée **chromosome** (ou individu), représente un **chemin complet.**



Un chromosome = un ordre de visite des villes, départ et retour en A.

Chromosome (individu) ▶

Chemin
A–B–C–D–A
A–B–D–C–A
A–C–B–D–A
A–C–D–B–A
A–D–B–C–A
A–D–C–B–A

} **POPULATION**

La sélection : choisir les parents

- La sélection est l'opérateur qui choisit les meilleures solutions de la population P pour former l'ensemble des **parents P'.**
- Elle s'appuie sur une **fonction d'évaluation (fitness)** qui mesure la qualité de chaque individu.

Technique	Principe
Roulette (roulette wheel)	Chaque individu a une chance proportionnelle à sa fitness (plus la distance est courte → plus la probabilité est élevée).
Tournoi	On tire au hasard un petit groupe d'individus et l'on garde le meilleur.
Rang	On classe les individus par performance et on leur attribue une probabilité selon leur position.

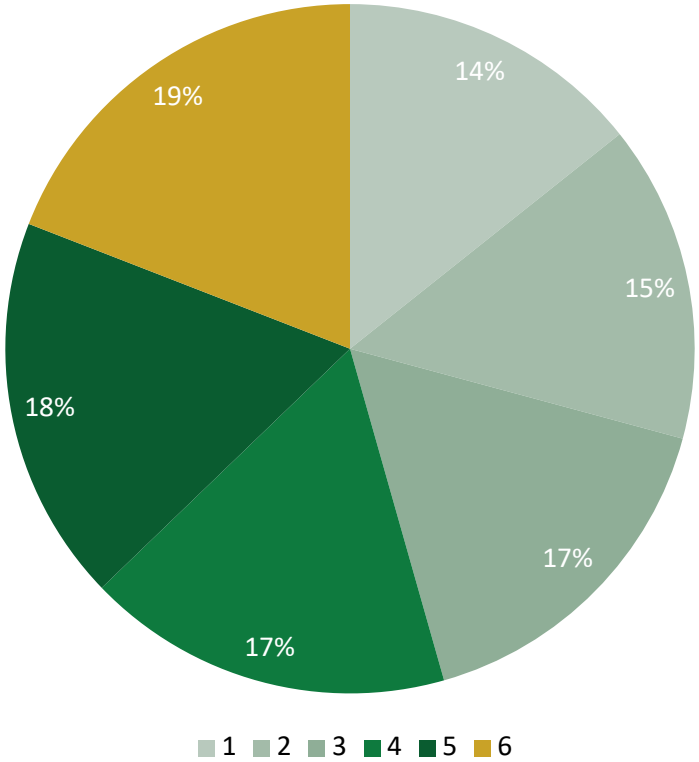
Sélection par roulette : un tirage proportionnel

Fitness(i) = 1 / distance(i) **P_i = Fitness(i) / Σ Fitness**

Individu	Chemin	Distance	Fitness	P(sélection)
1	A–B–C–D–A	120	0,00833	14,3 %
2	A–B–D–C–A	115	0,00870	14,9 %
3	A–C–B–D–A	105	0,00952	16,4 %
4	**A–C–D–B–A**	**100**	**0,01000**	**17,2 %**
5	**A–D–B–C–A**	**95**	**0,01053**	**18,1 %**
6	**A–D–C–B–A**	**90**	**0,01111**	**19,1 %**

Individus sélectionnés : 4, 5 et 6 (les plus probables) → ils serviront de parents.

La roue de la roulette



Évaluer un chromosome avec une matrice de distances

	A	B	C	D
A	0	20	35	42
B	20	0	30	34
C	35	30	0	12
D	42	34	12	0

La fitness d'un chromosome se calcule en additionnant les distances de ses arêtes, lues dans la matrice.

À VOUS DE CALCULER

Quelle est la distance du chromosome A–B–C–D–A ?

20 + 30 + 12 + 42 = 104

Et A–C–B–D–A ?

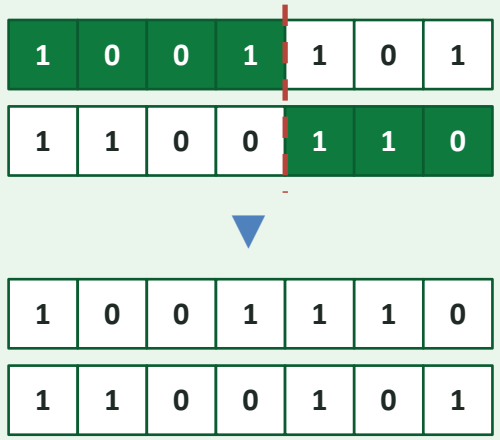
35 + 30 + 34 + 42 = 141 → moins bon (fitness plus faible)

Le croisement : recombiner les gènes des parents

Le croisement construit une ou deux nouvelles solutions (enfants) à partir de deux parents. Il est appliqué avec une probabilité, en général comprise entre 65 % et 95 %.

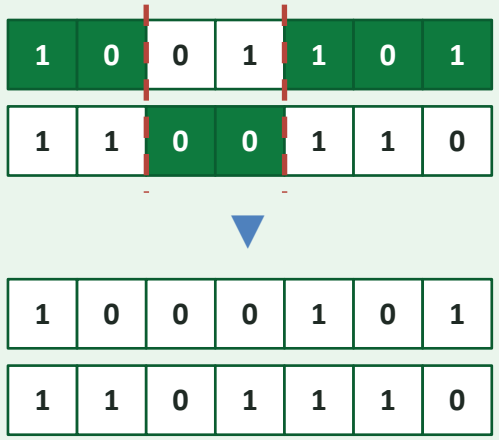
À un point

Découpe les parents en deux parties par un point choisi au hasard.



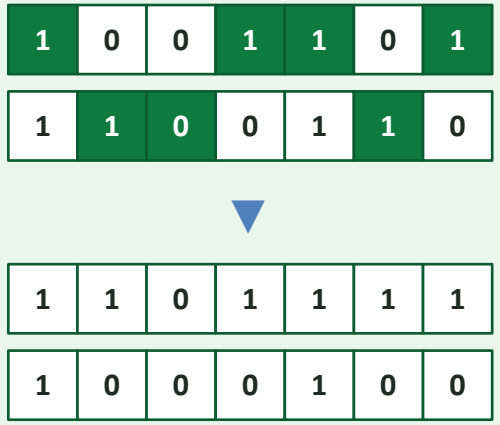
À deux points

Découpe les parents en trois parties par deux points choisis au hasard.



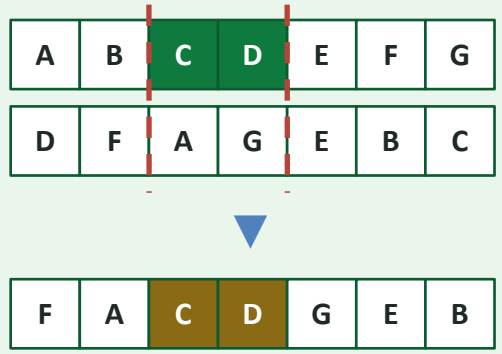
Uniforme

Pour chaque gène, on tire au hasard le parent qui le transmet.



OX (ordonné)

Copie un segment d'un parent, complète avec les gènes de l'autre dans leur ordre, sans doublon.



Croisement OX appliqué au TSP

Parent	Chemin	Distance
P1	A – D – B – C – A	95
P2	A – D – C – B – A	90
P3	A – C – D – B – A	100

Croisement	Parents utilisés	Segment conservé	Enfant obtenu (OX)
1	P1 × P2	la ville D (de P1)	A – D – C – B – A
2	P2 × P3	la ville C (de P2)	A – D – C – B – A
3	P3 × P1	la ville D (de P3)	A – B – D – C – A

Règle : on garde la ville du segment à sa position, puis on complète avec l'ordre de l'autre parent, sans doublon.

La mutation : une petite modification aléatoire

- La mutation apporte une petite modification aléatoire à la solution, par exemple en altérant un ou deux gènes.
- Son but : assurer une bonne exploration de l'espace de recherche (éviter que toute la population se ressemble).
- Elle est appliquée avec une **faible probabilité**, en général entre 1 % et 5 %.



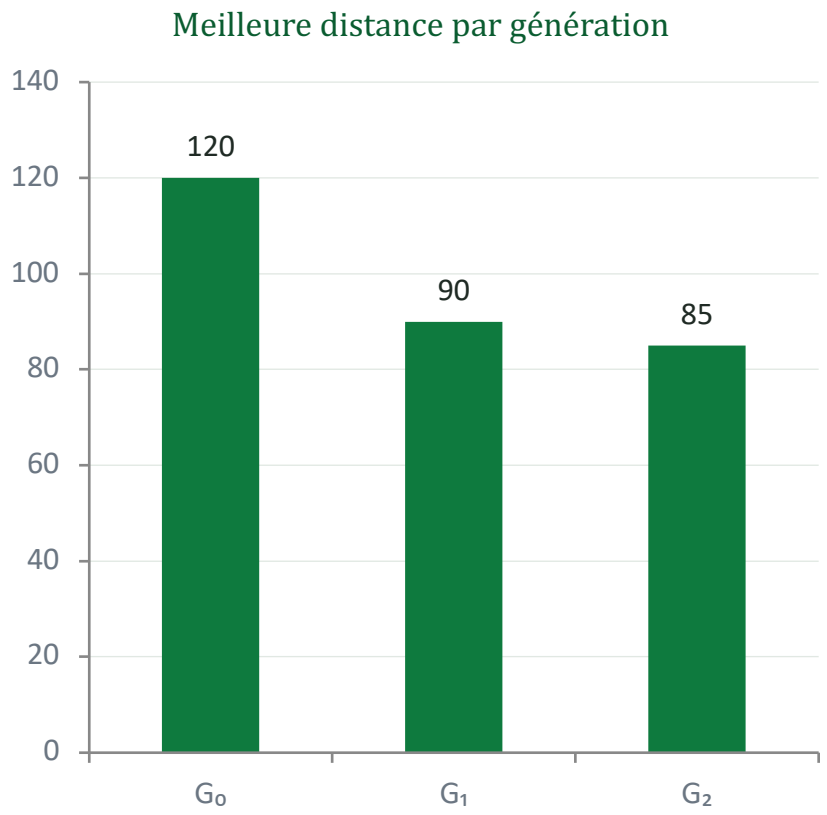
le 3ᵉ gène passe de 0 à 1

Mutation dans le TSP et évolution des générations

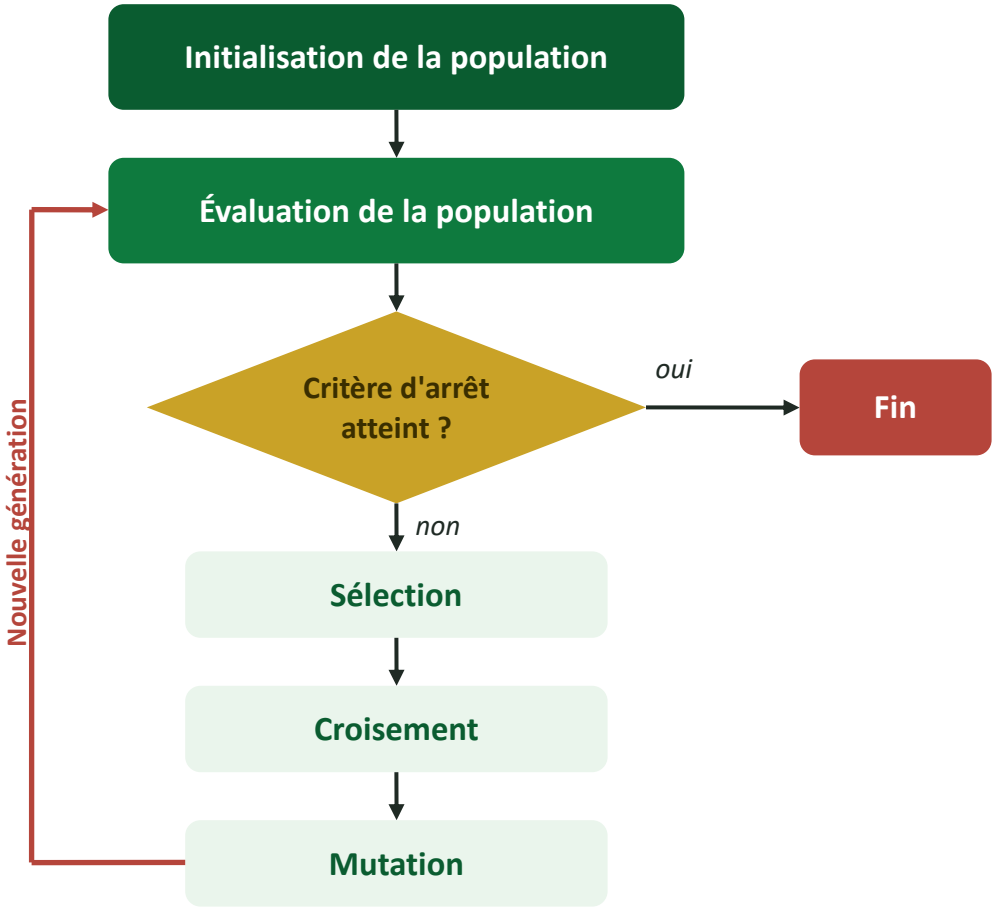
Dans le TSP, la mutation correspond à un échange ou une inversion de villes dans un chemin.

Type de mutation	Principe	Exemple
Échange (swap)	Échange deux villes au hasard.	A–D–C–B–A → A–C–D–B–A
Insertion	Déplace une ville à une autre position.	A–B–C–D–A → A–C–B–D–A
Inversion	Inverse l'ordre d'un sous-ensemble de villes.	A–B–C–D–A → A–D–C–B–A

Génération	Meilleur individu	Distance
G_0 (initiale)	A–B–C–D–A	120
G_1 (sélection + croisement)	A–D–C–B–A	90
G_2 (après mutation)	A–D–B–C–A	85



L'algorithme génétique complet




```
1  Initialisation : créer une population aléatoire
2  Répéter jusqu'au critère d'arrêt :
3     1. Évaluer la fitness de chaque individu
4     2. Sélectionner les parents
5     3. Appliquer le croisement
6     4. Appliquer la mutation
7     5. Former la nouvelle génération
         (remplacement)
8  Retourner la meilleure solution trouvée
```

Quand arrêter l'algorithme génétique ?

L'AG ne s'arrête pas de lui-même : on définit une condition d'arrêt, compromis entre temps d'exécution et qualité de la solution.

Critère	Description	Exemple dans le TSP
Nombre maximal de générations	On s'arrête après un nombre fixé d'itérations.	100 générations
Stagnation de la fitness	La meilleure solution ne s'améliore plus depuis un certain nombre de générations.	La distance reste ≈ 85 pendant 10 générations
Seuil de performance atteint	La solution atteint une valeur jugée satisfaisante, fixée à l'avance.	Distance < 80 km
Limite de temps	On fixe un temps maximal d'exécution.	30 secondes
Diversité faible	Toute la population devient identique (perte de variété).	Tous les chemins sont les mêmes

QUIZ · VRAI OU FAUX ?

Un algorithme génétique garantit toujours de trouver le circuit optimal du TSP.

Vrai

Faux

Réponse Faux : c'est une métaheuristique. Elle trouve rapidement une bonne solution approchée, sans garantie d'optimalité.

TD

TRAVAUX DIRIGÉS 3

Algorithme génétique et TSP

Faire évoluer une population de circuits

Le voyageur de commerce à 5 villes

Objectif : trouver un chemin optimal permettant au voyageur de visiter chaque ville exactement une fois et de revenir à sa ville de départ (ville 1).

- Générez aléatoirement une population initiale de 4 chromosomes.
- Calculez la distance et la fitness (1/distance) de chacun.
- Calculez les probabilités de la roulette et sélectionnez 2 parents.
- Appliquez un croisement OX, puis une mutation par échange.
- Comparez le meilleur enfant au meilleur parent. Quel critère d'arrêt choisiriez-vous ?

	1	2	3	4	5
1	×	15	3	10	11
2	15	×	5	6	2
3	3	5	×	8	7
4	10	6	8	×	3
5	11	2	7	3	×

Matrice des distances entre les 5 villes

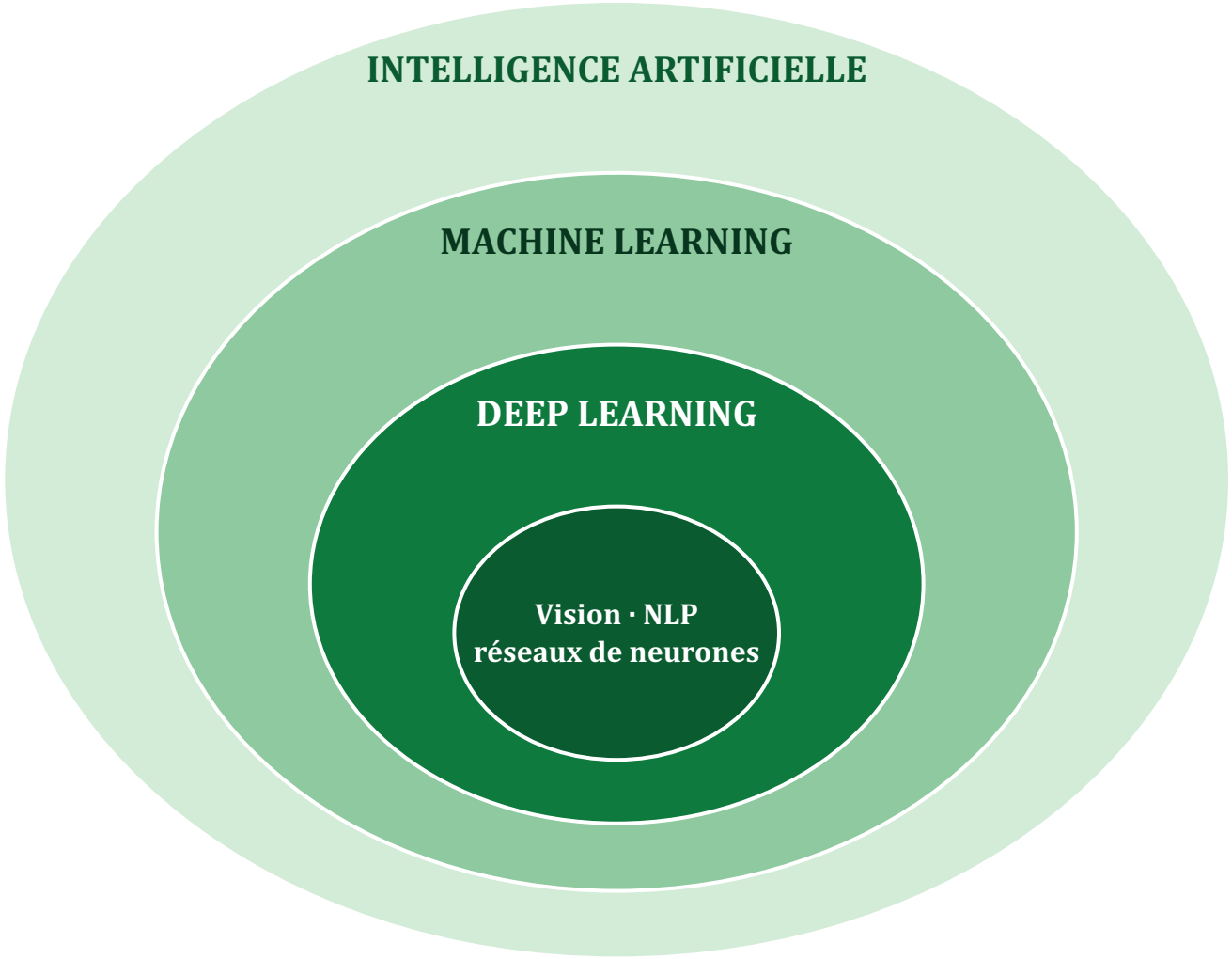
SECTION 5

L'apprentissage automatique (Machine Learning)

« Apprendre plutôt que programmer »

IA, Machine Learning, Deep Learning : des ensembles emboîtés

- **L'IA :** le domaine qui englobe tout (y compris les systèmes à base de règles).
- **Le Machine Learning (ML) :** la méthode moderne où l'ordinateur apprend ses propres règles à partir des données, sans être programmé spécifiquement pour chaque tâche.
- **Le Deep Learning (DL) :** le ML spécialisé qui utilise des réseaux de neurones profonds ; essentiel pour les données complexes.
- **Vision par ordinateur et NLP :** deux applications majeures du DL (chapitre 3).



Le Machine Learning : apprendre à partir des données

Définition. Le machine learning est une branche de l'IA qui consiste à donner à une machine la capacité d'apprendre plutôt que de la programmer de façon explicite.



modèle prédictif ou organisateur

Programmation classique

Règles + Données → Réponses

le développeur écrit les règles « si… alors… »

Machine Learning

Données + Réponses → Règles

la machine déduit les règles à partir d'exemples

Les données : la matière première de l'apprentissage

Les données sont l'ensemble des **observations, mesures et exemples** que l'on collecte pour permettre à une machine d'apprendre. Elles constituent la **base de connaissances** à partir de laquelle le modèle va apprendre.

Données non structurées

textes, images, sons, vidéos…



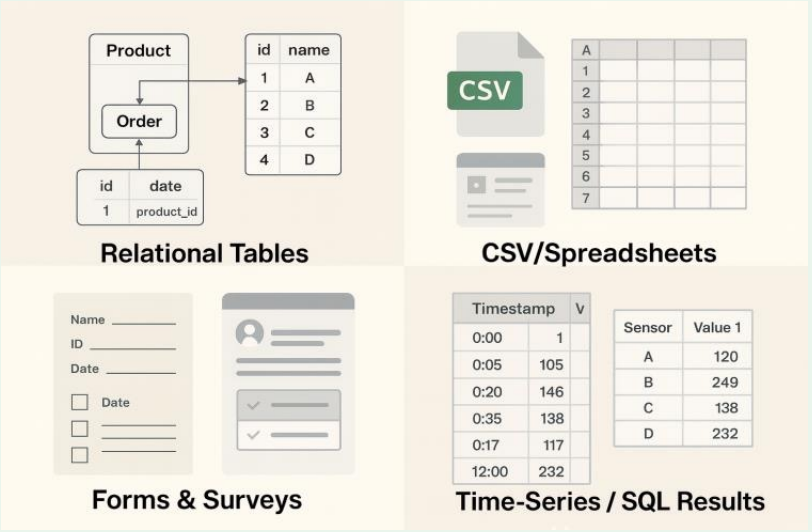




Texte · Vidéo · Image

Données structurées

tables relationnelles, feuilles de calcul, formulaires, séries temporelles



Données libellées et non libellées

Données non libellées


→ ?


→ ?


→ ?

Données libellées


→ chat

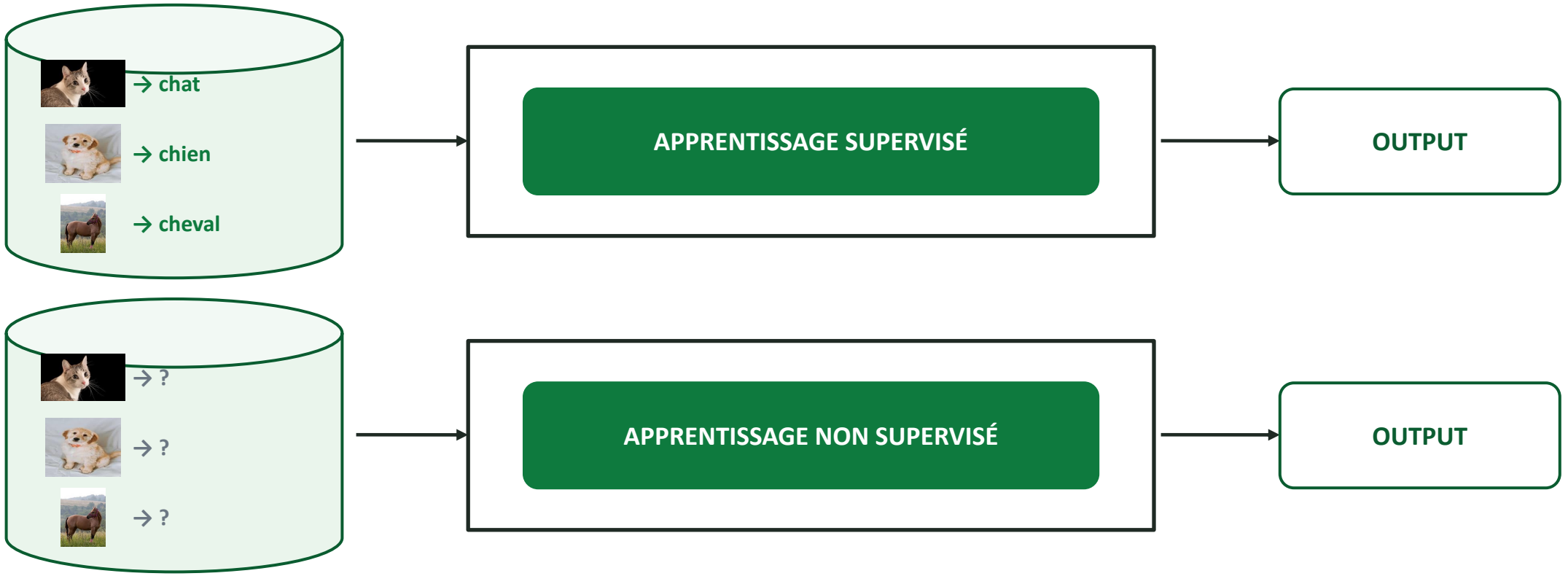

→ chien


→ cheval

Le libellé (ou étiquette) est la réponse attendue y associée à chaque exemple x.

L'algorithme de ML : supervisé ou non supervisé

Un **algorithme de Machine Learning** est le processus qui permet à une machine d'apprendre à partir des données afin de construire un **modèle prédictif ou organisateur.**



L'apprentissage supervisé : un professeur corrige

L'apprentissage supervisé est l'approche où l'on supervise l'apprentissage de la machine en lui fournissant, pour chaque exemple d'entrée (x), la sortie attendue (y). C'est comme si un « professeur » (le y) corrigeait constamment l'algorithme.

Classification

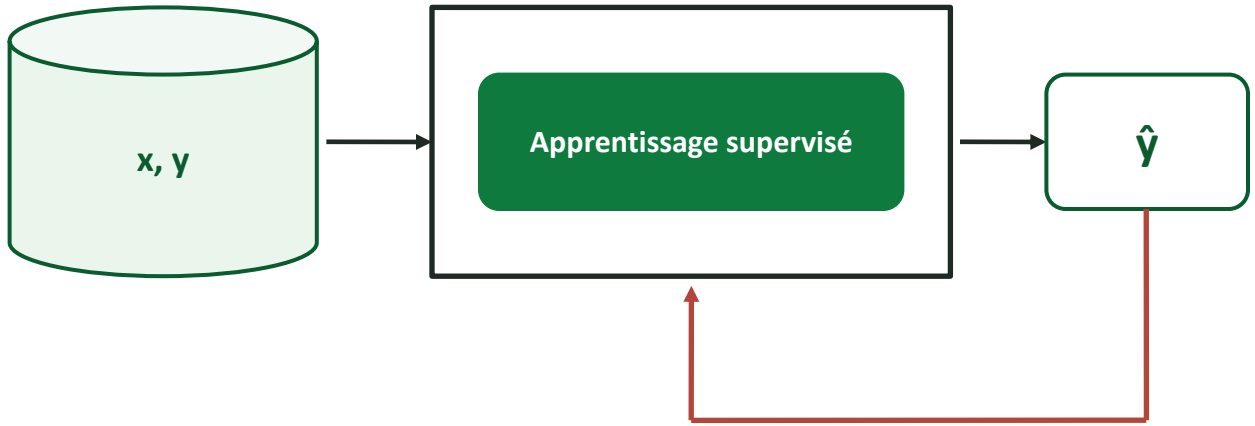
prédire une classe (catégorie)

ex. chat / chien / cheval · spam / non-spam

Régression

prédire une valeur numérique continue

ex. prix d'une maison · température

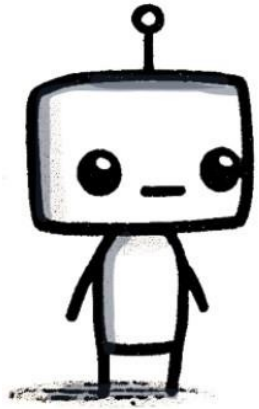


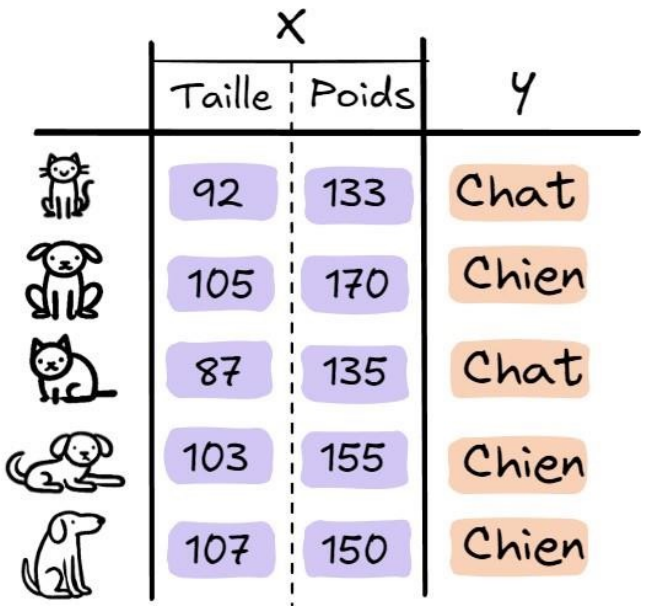
correction : on compare ŷ au vrai y et l'on ajuste le modèle

Classification : apprendre à distinguer un chat d'un chien

La classification sert à **prédire une classe**. Pour apprendre à reconnaître un chien ou un chat à partir de sa taille et de son poids, on fournit à la machine un jeu de données (X, y) afin qu'elle s'entraîne.

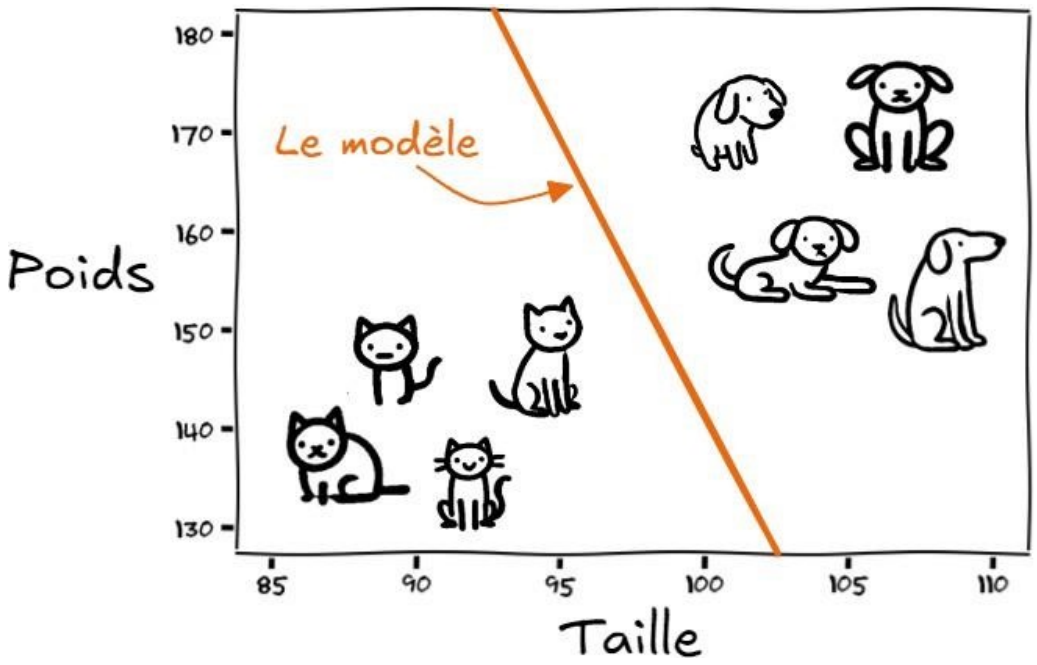
- **X (caractéristiques) :** taille, poids
- **y (étiquette) :** « chat » ou « chien »
- Chaque ligne du tableau = un exemple d'entraînement





Entraînement : trouver la frontière entre les classes

En étudiant ces données, la machine trouve le modèle qui sépare au mieux les deux classes de points, c'est-à-dire celui qui ne commet aucune erreur sur le jeu de données qui lui a été présenté.



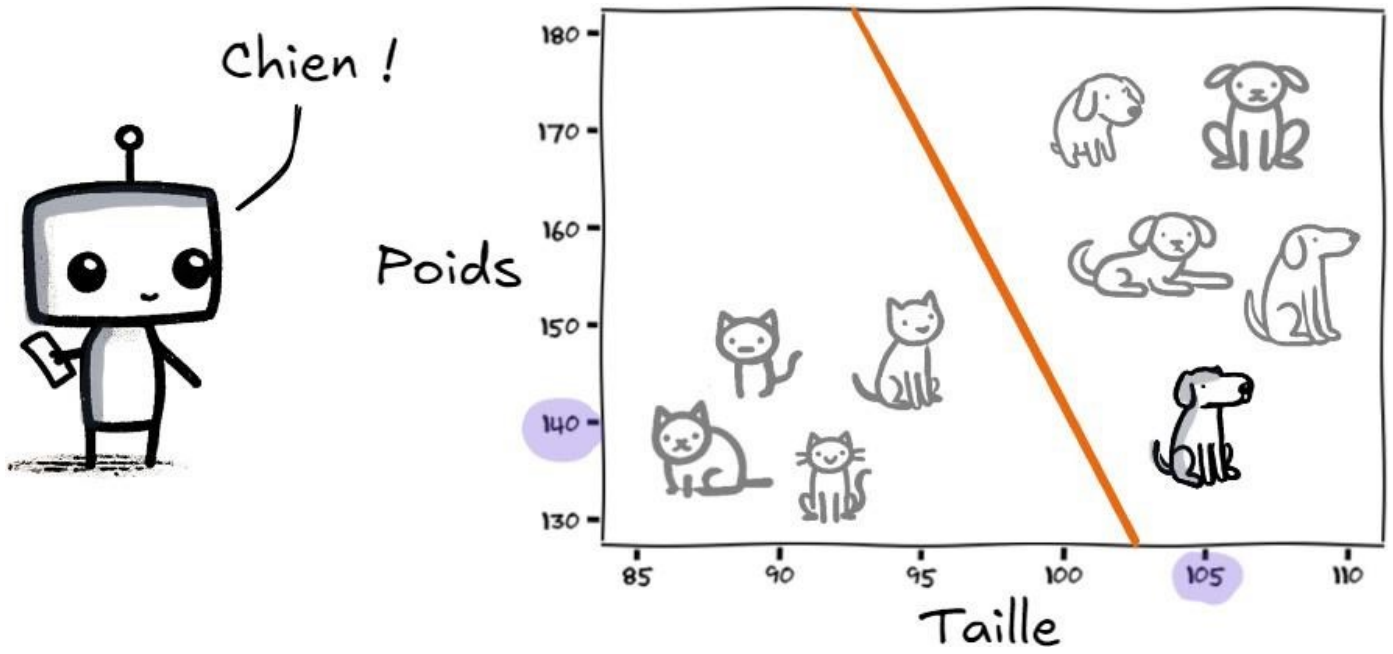


DONNÉES → ALGORITHME DE CLASSIFICATION (ENTRAÎNEMENT)

Prédiction : classer un animal jamais vu

Après l'entraînement, le modèle peut faire de nouvelles prédictions. Voici un animal qu'il n'a jamais vu auparavant :





D'après le modèle, cet animal (taille 105, poids 140) se situe du côté des chiens.

La machine prédit donc : « chien ».

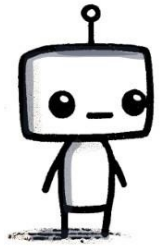
ALGORITHME DE CLASSIFICATION · ÉVALUATION DU MODÈLE

Régression : prédire le prix d'une maison

Un modèle de régression correspond aux situations où la machine doit prédire la valeur d'une **variable quantitative** (continue).

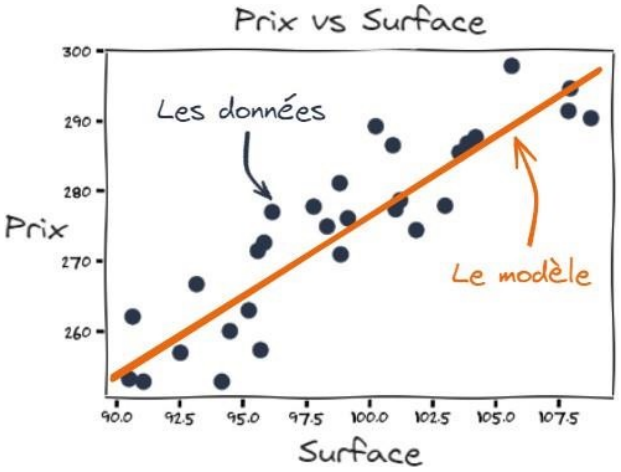


Régression → 252 000



	X	y
	92	259K
	95	237K
	98	272K
	93	234K
	100	252K
	110	300K

Grâce au jeu de données (surface X, prix y), la machine apprend un modèle permettant de prédire y en fonction de X. Elle s'auto-évalue en cherchant le modèle qui offre les meilleures performances sur les données fournies.

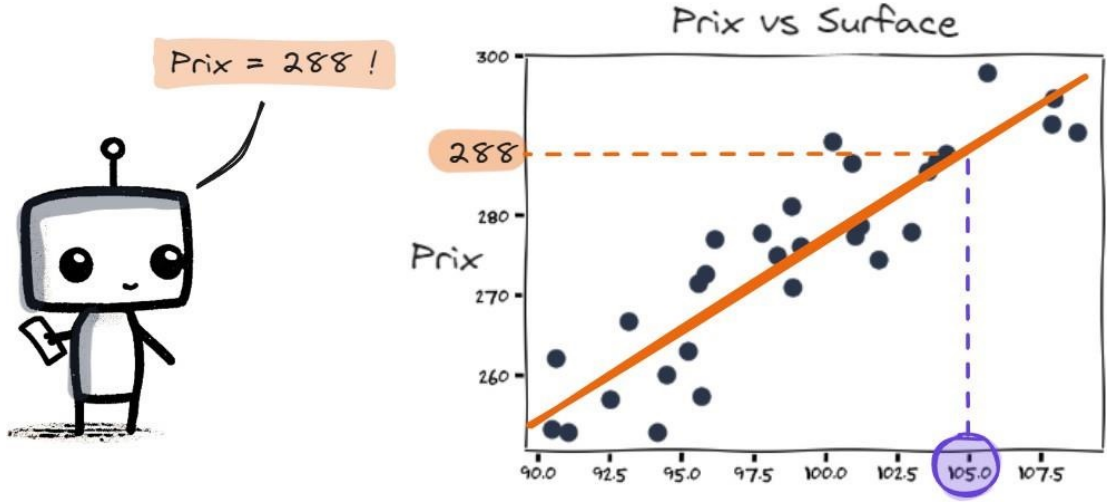




Utiliser le modèle de régression

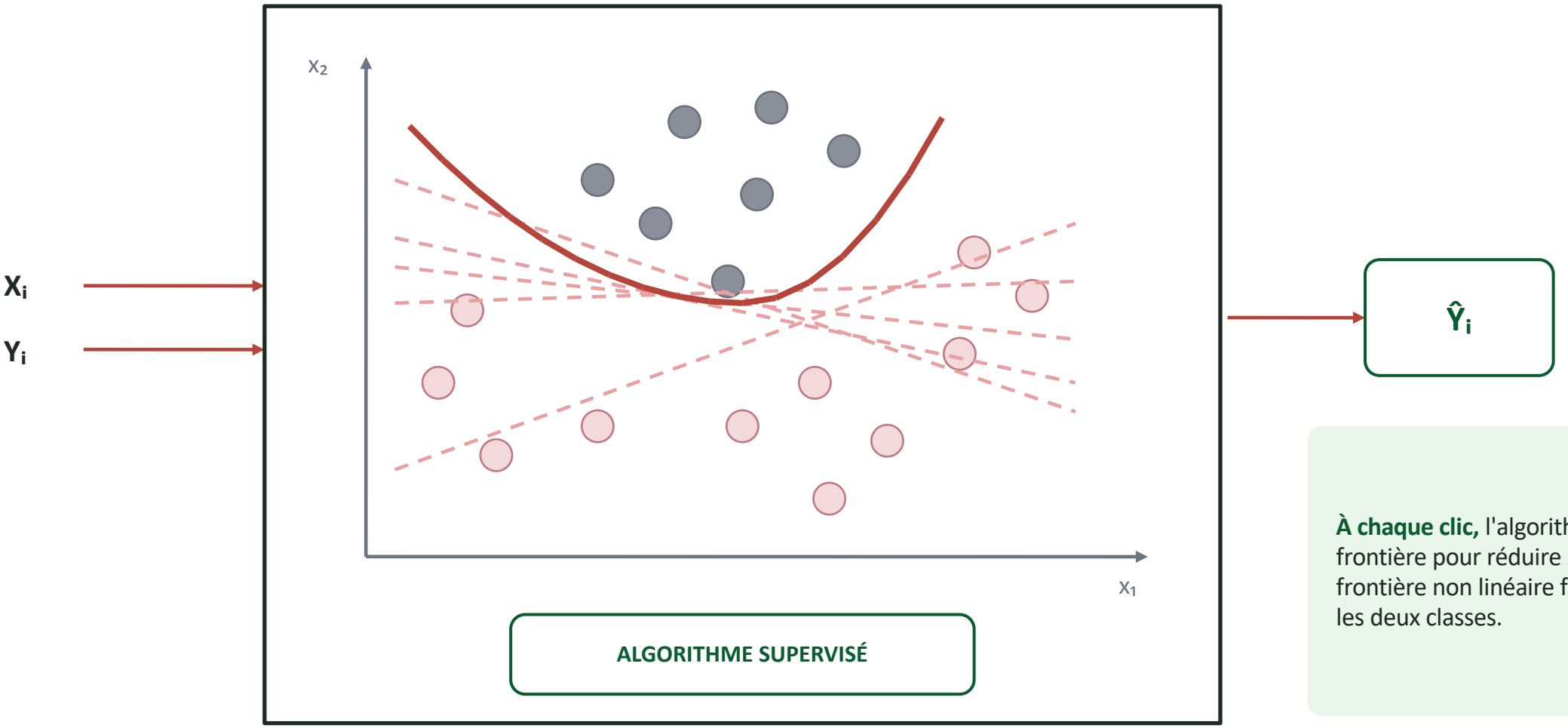
Une fois le modèle entraîné sur le jeu de données, on peut s'en servir pour faire de futures prédictions. Par exemple, pour une nouvelle maison de 105 m² de surface habitable :





Le modèle prédit le prix de cette nouvelle maison : environ 288 k.

Comment l'algorithme ajuste son modèle, essai après essai



À chaque clic, l'algorithme corrige sa frontière pour réduire ses erreurs ; une frontière non linéaire finit par séparer les deux classes.

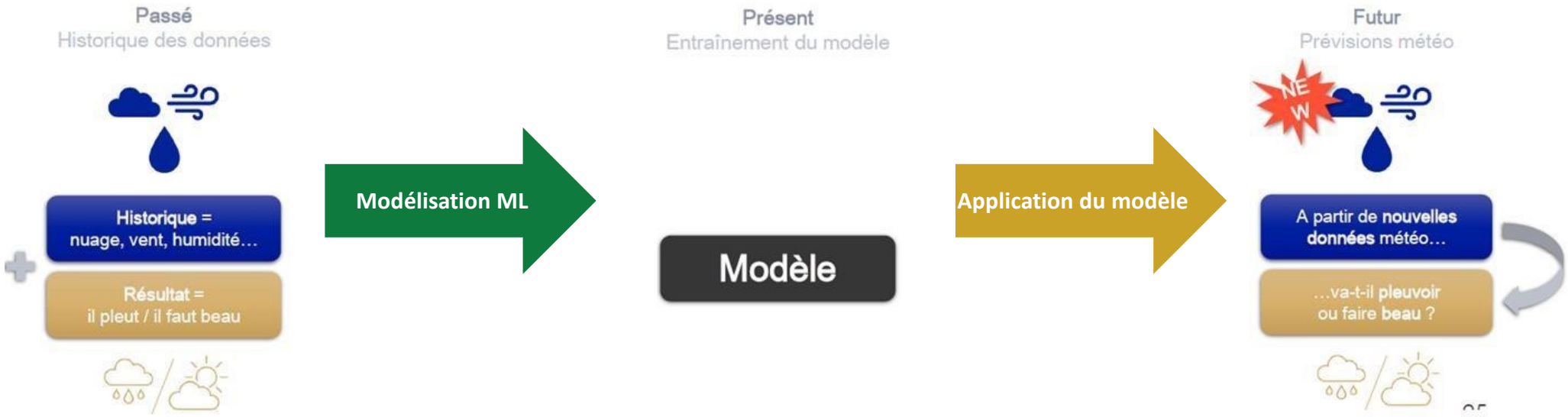
Exemple : prévoir la pluie à partir de mesures

X (caractéristiques)

Nébulosité	Humidité	Température	Pression
0,5	80 %	75 °F	1,2

Y (étiquette)

Pluie ?
1 (oui)



Passé : historique des données

Présent : entraînement du modèle

Futur : prévisions

QUIZ · CLASSIFICATION OU RÉGRESSION ?

Lesquels de ces problèmes sont des problèmes de régression ?

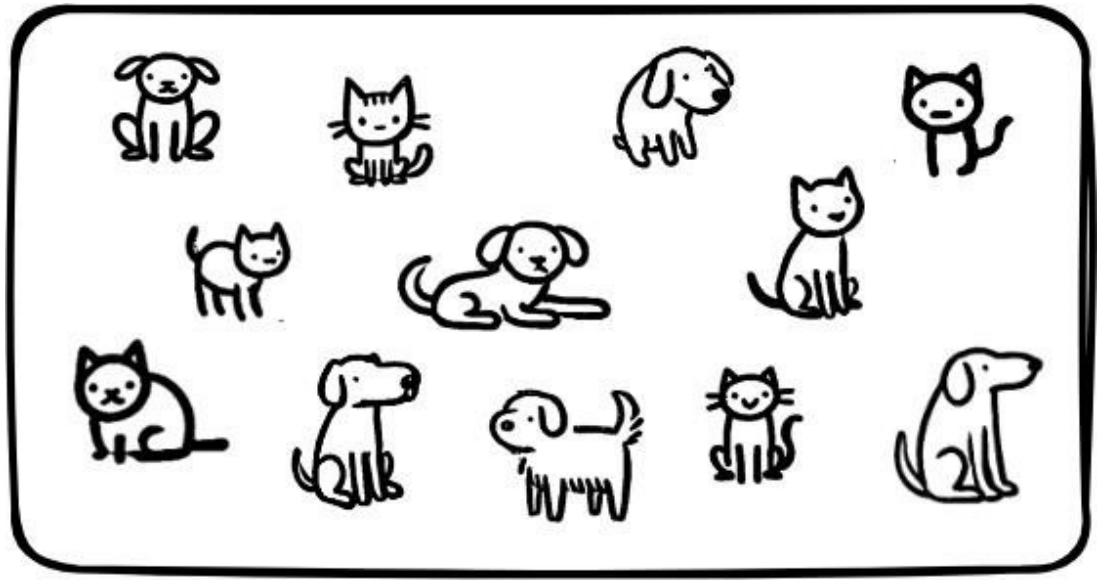
- **Prix d'un appartement**
- **Courriel spam ou non**
- **Température de demain**
- **Chat ou chien**

Réponse Régression : prix d'un appartement et température de demain (valeurs continues). Classification : spam / non-spam et chat / chien (classes).

L'apprentissage non supervisé : trouver la structure

C'est la seconde grande branche du ML. Contrairement au supervisé, elle laisse la machine **découvrir par elle-même des structures** présentes dans les données, sans relation entrée → sortie (x → y) à apprendre.

Exemple : on présente les animaux vus précédemment sans préciser s'il s'agit de chats ou de chiens, et on demande simplement de les regrouper selon leur ressemblance. On fournit uniquement les attributs X, sans la sortie y.

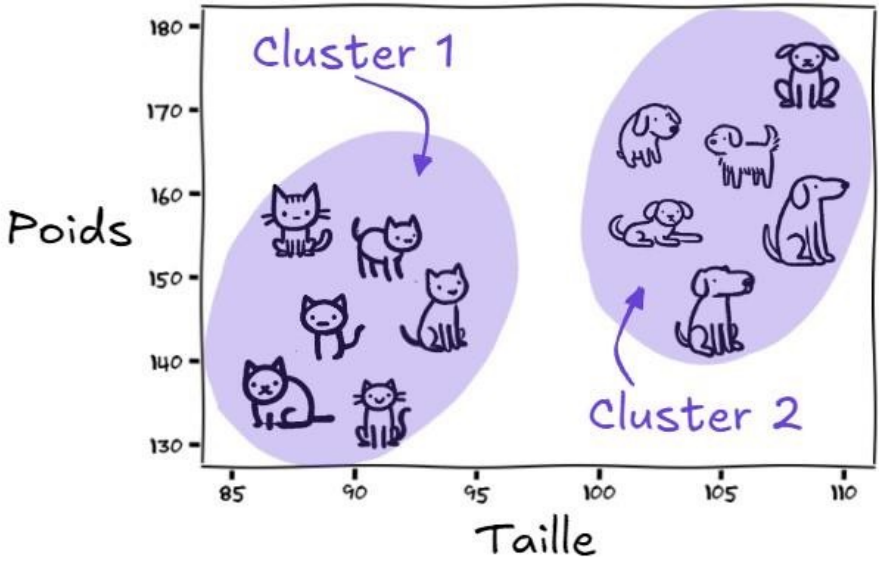


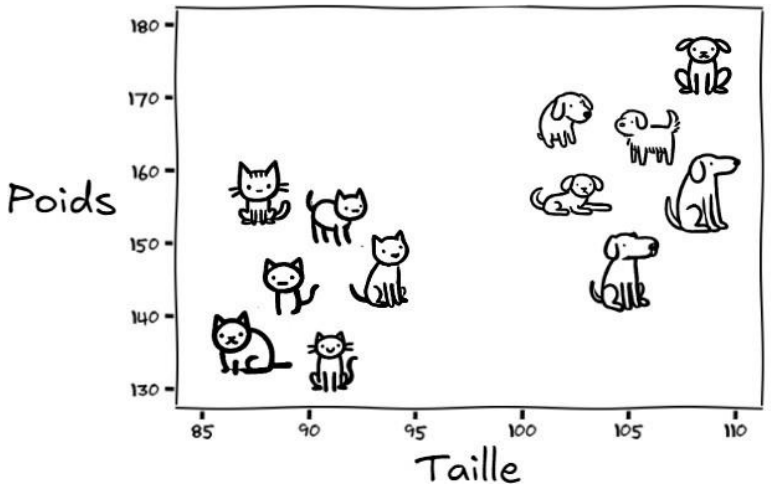
Principale application : le clustering (regroupement).

Le clustering : regrouper sans étiquettes

En examinant les seuls attributs X (taille, poids, apparence…), la machine remarque d'elle-même que certains animaux se ressemblent.









Elle peut alors les regrouper, sans pour autant savoir s'il s'agit de chats ou de chiens : on ne l'a pas contrainte à apprendre ce genre de chose.

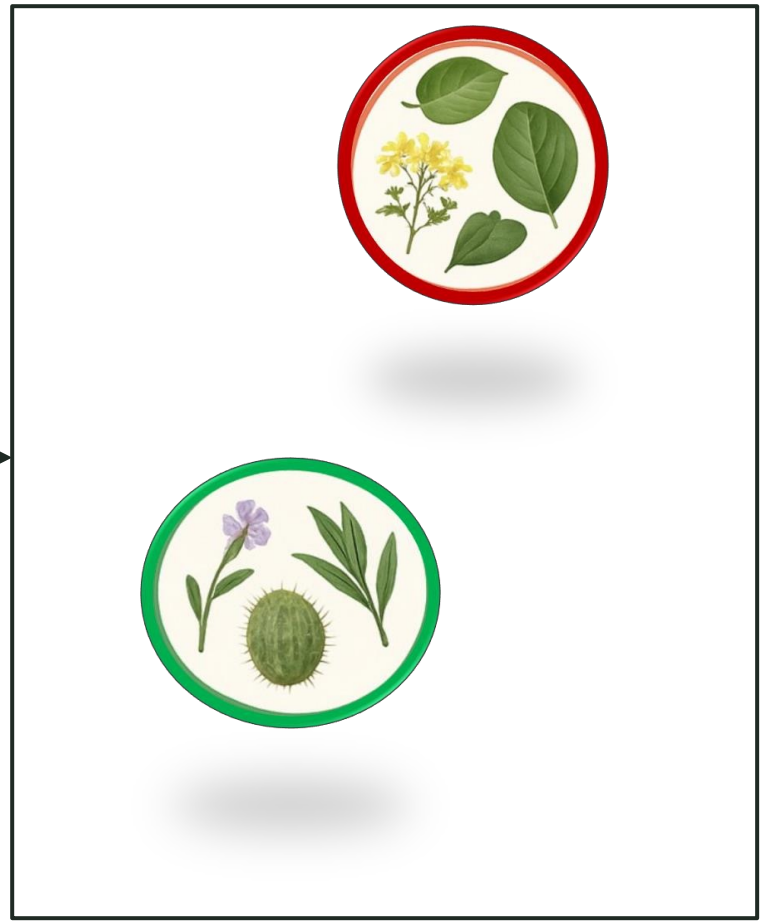
Ce type d'application porte le nom de clustering.

Exemple de clustering : trier des feuilles avec K-means

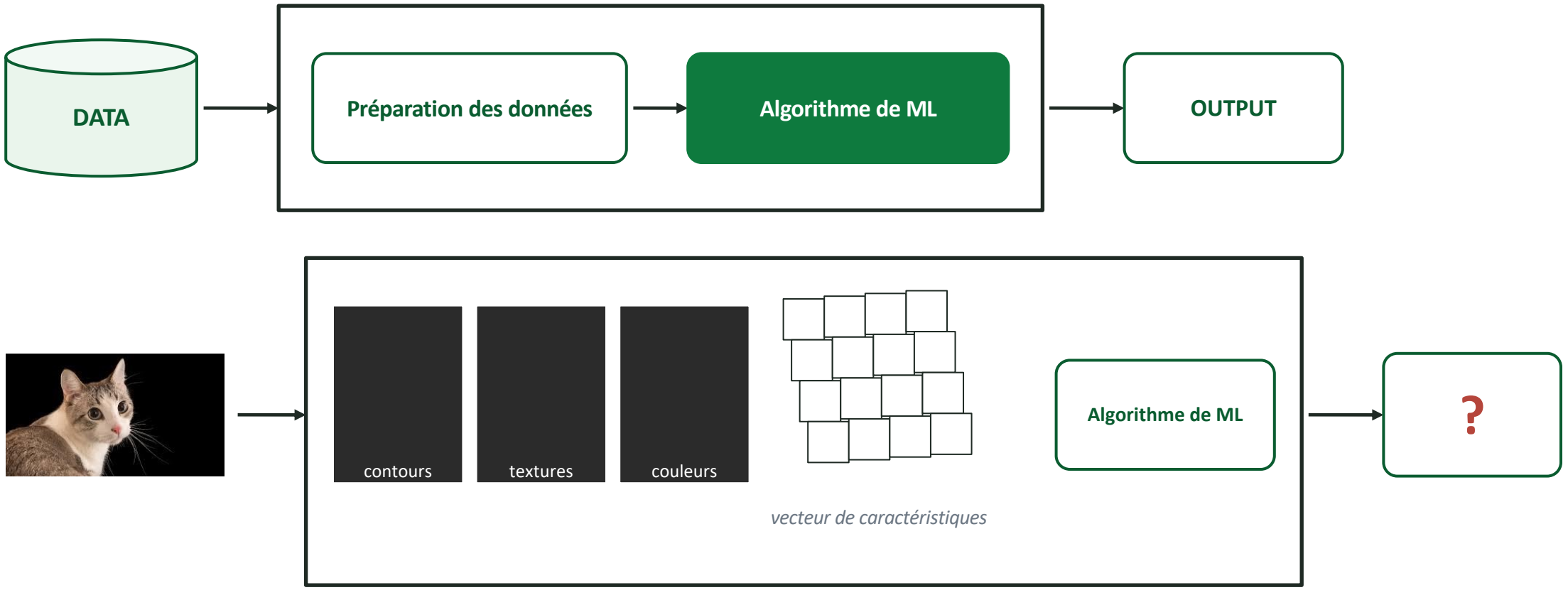


K-means

Avant K-means, on applique souvent une ACP (PCA) pour réduire le nombre de caractéristiques.



Les limites du ML classique : préparer les données à la main



Pour une image, un son ou un texte, l'expert doit concevoir lui-même les caractéristiques : long, coûteux et souvent insuffisant.

SECTION 6

Deep Learning et réseaux de neurones

« S'inspirer du cerveau »

Le Deep Learning : apprendre les caractéristiques elles-mêmes

Le Deep Learning (apprentissage profond) n'est pas une technologie en marge : c'est une **sous-catégorie spécialisée et particulièrement puissante du Machine Learning**. Il se distingue par l'utilisation de **réseaux de neurones artificiels (RNA) à structure profonde.**

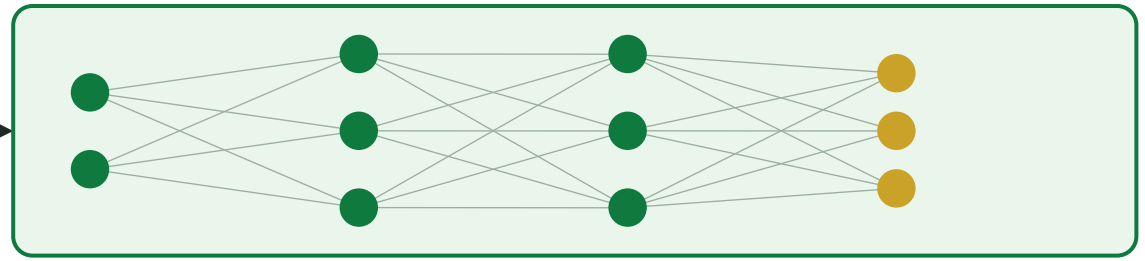
Machine Learning



Extraction manuelle des caractéristiques → Classification (ML) → « chat »

Deep Learning





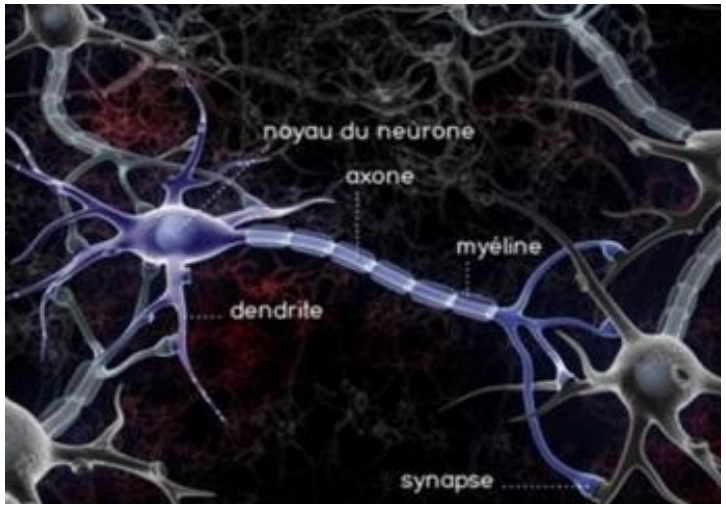
« chat »

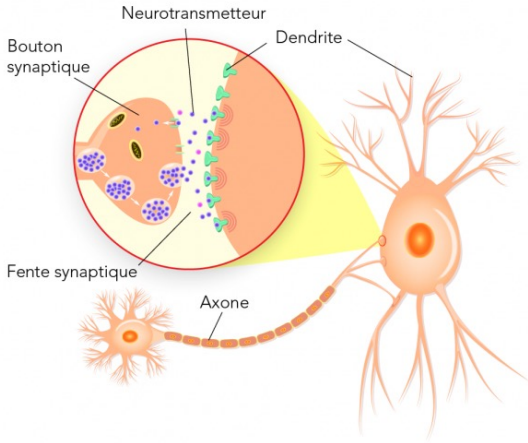
extraction + classification apprises ensemble

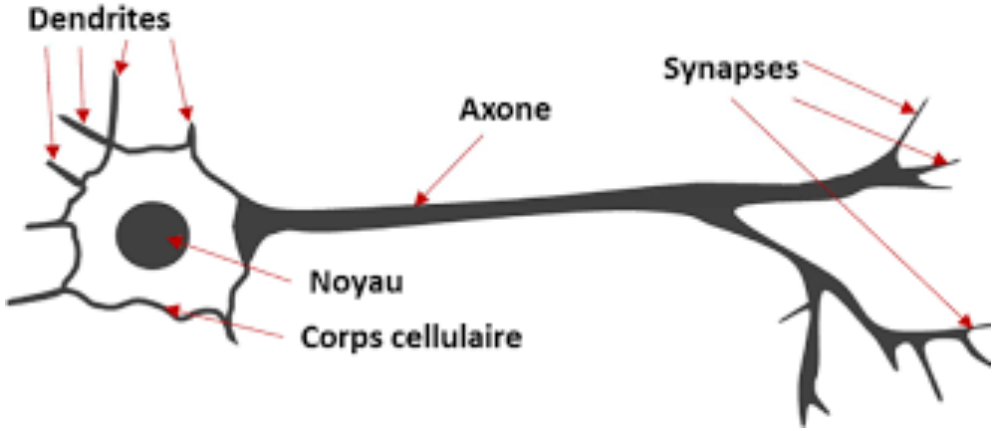
Le réseau apprend seul, couche après couche, les caractéristiques utiles.

Le neurone biologique : la source d'inspiration

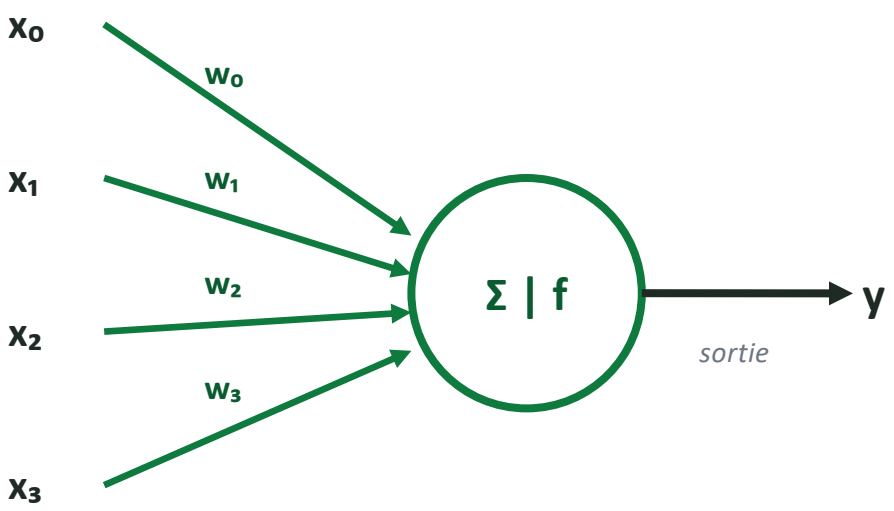
- **Dendrites :** les « antennes » du neurone ; elles reçoivent les signaux.
- **Corps cellulaire (soma) :** le centre de commande ; il agrège et somme les signaux électriques et chimiques reçus des autres neurones.
- **Synapses :** les points de connexion entre l'axone d'un neurone et la dendrite d'un autre ; elles modulent la force (l'importance) du signal transmis.
- **Axone :** le « câble de sortie » ; si le signal cumulé dépasse un seuil, il transmet l'information aux autres neurones via ses terminaisons.







Le neurone artificiel : une modélisation mathématique



Neurone biologique	Neurone artificiel
Dendrites	Entrées x_i
Synapses (force)	Poids w_i (excitateur > 0, inhibiteur < 0)
Corps cellulaire	Somme pondérée f (avec biais b)
Seuil de déclenchement	Fonction d'activation
Axone	Sortie y

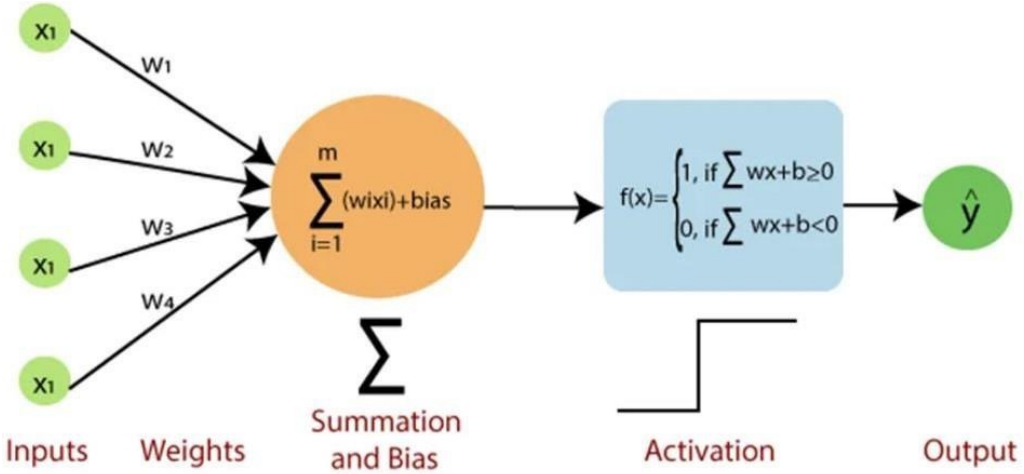
Agrégation : $f = x_1w_1 + x_2w_2 + x_3w_3 + b$

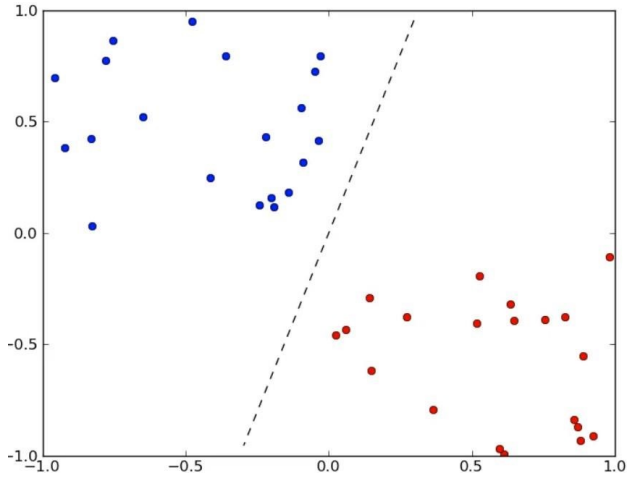
Activation (seuil) : $y = 1$ si $f > 0$, sinon $y = 0$

Modèle formel de McCulloch et Pitts (1943).

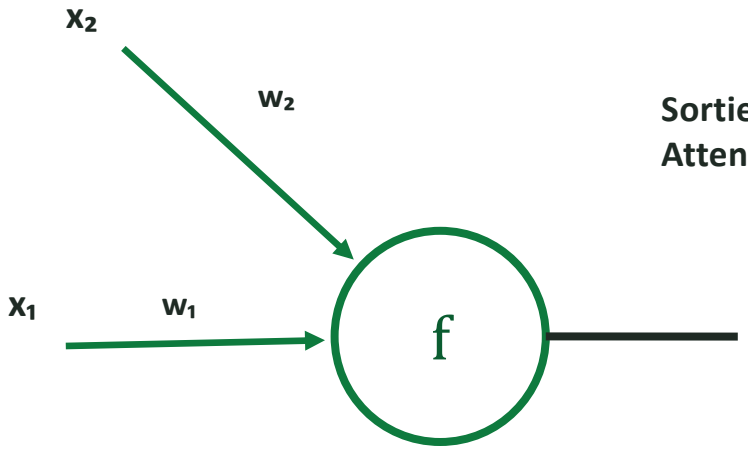
Le perceptron : le premier neurone qui apprend

- **Le perceptron (Rosenblatt, 1958)** est le modèle historique qui a concrétisé l'idée d'un neurone artificiel inspiré par la biologie : l'unité de construction la plus simple des réseaux de neurones.
- Il reçoit plusieurs **signaux d'entrée** (les x). Chaque signal est d'abord **pondéré** (multiplié par un poids w) pour refléter son importance, comme la force des synapses.
- Les entrées pondérées sont **additionnées**, puis passées dans une fonction d'activation qui produit un **signal de sortie unique.**

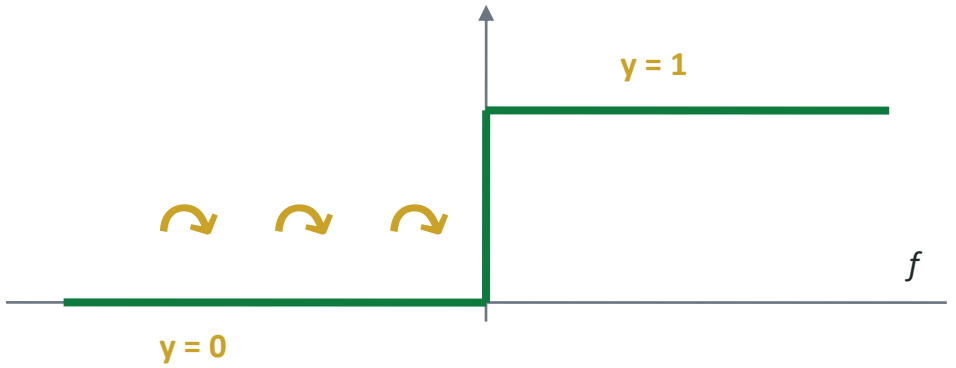




Entraîner le perceptron : la règle d'apprentissage



Sortie y = 0
Attendu y_vrai = 1



l'apprentissage augmente f jusqu'à franchir le seuil

Règle de mise à jour :

$$\mathbf{w \leftarrow w + \alpha\,(y_vrai - y)\cdot x}$$

ici : $w \leftarrow w + \alpha\,(1 - 0)\cdot x = w + \alpha \cdot x$

α : taux d'apprentissage

Avec $x_1 = 1$ et $x_2 = 0$:

$w_1 \leftarrow w_1 + \alpha$ → **renforcement**

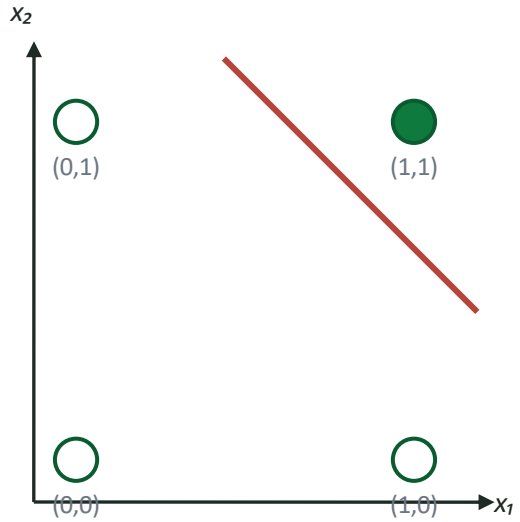
$w_2 \leftarrow w_2$ ($x_2 = 0$: pas de changement)

Seuls les poids des entrées actives sont corrigés.

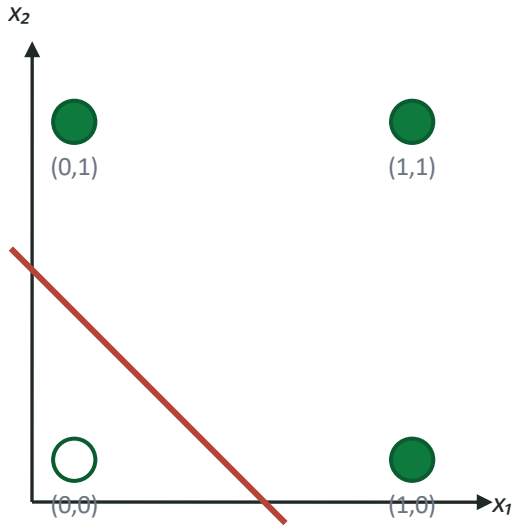
Ce que sait faire le perceptron : séparer linéairement

Le perceptron sépare **linéairement** des classes de données. Il fonctionne très bien pour des fonctions logiques **linéairement séparables** comme ET (AND) et OU (OR) : une seule droite suffit à séparer les sorties 0 et 1.

x_1	x_2	ET
0	0	0
0	1	0
1	0	0
1	1	1



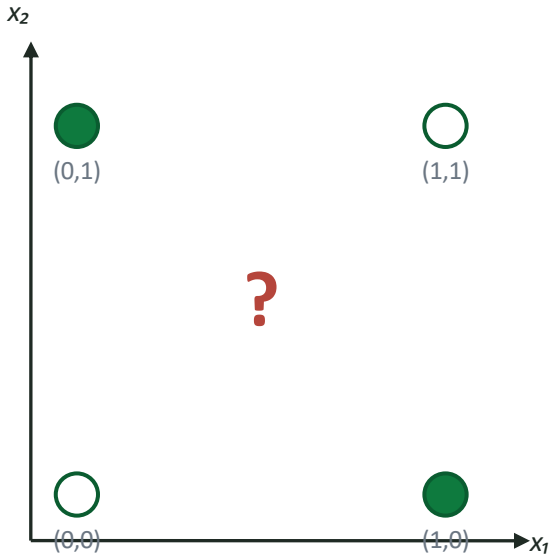
x_1	x_2	OU
0	0	0
0	1	1
1	0	1
1	1	1



● *vert = sortie 1* ○ *blanc = sortie 0* — *droite de séparation*

La limite du perceptron : le problème du XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



Aucune droite ne sépare les points verts des points blancs.

Le perceptron ne peut pas résoudre le XOR car il est limité aux frontières de décision linéaires (Minsky et Papert, 1969).

Pour traiter ce type de problème, il faut introduire :

- plusieurs couches de neurones → **perceptron multicouche (MLP)**
- des fonctions d'activation non linéaires → **sigmoïde, ReLU, tanh…**

Cette limite a contribué au premier « hiver de l'IA » (chapitre 1) ; la rétropropagation (1986) a relancé les réseaux multicouches.

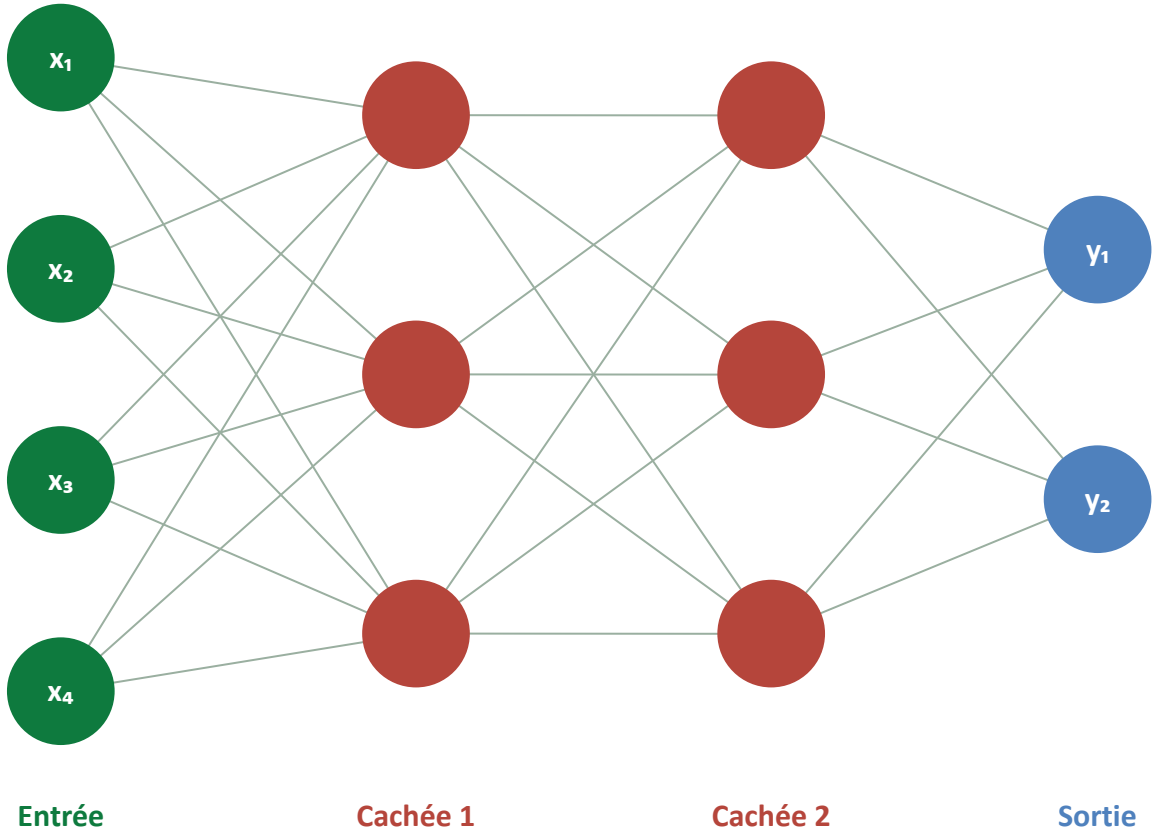
Le réseau de neurones artificiels

- Un réseau de neurones n'est qu'un **regroupement de composantes de base**, les neurones (perceptrons), interconnectées entre elles.
- Les neurones sont organisés en trois types de couches :

✓ **Couche d'entrée** — *input layer*

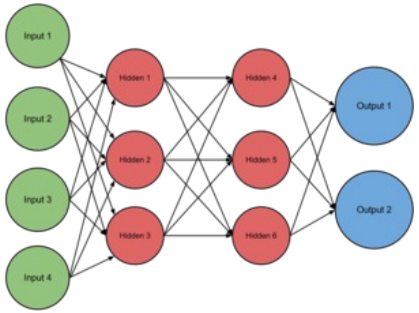
✓ **Couche(s) cachée(s)** — *hidden layers*

✓ **Couche de sortie** — *output layer*



Différentes architectures pour différents problèmes

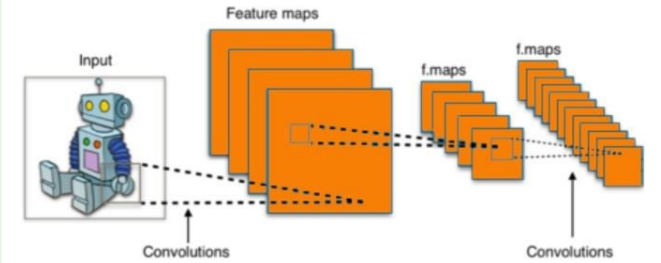
On catégorise les réseaux selon leur architecture : la manière dont les neurones sont reliés entre eux.



Réseau à propagation avant (feed-forward)

Perceptron multicouche : l'information circule de l'entrée vers la sortie.

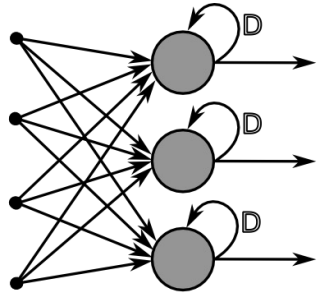
Usage : données tabulaires, scoring



Réseau convolutif (CNN)

Des filtres de convolution extraient des motifs locaux (bords, formes).

Usage : images, vision par ordinateur



Réseau récurrent (RNN)

Des boucles conservent une mémoire des entrées précédentes.

Usage : séquences : texte, parole, séries temporelles

Les réseaux profonds au service des données complexes

Le Machine Learning traditionnel excelle sur les données structurées (tableaux, chiffres). La puissance des réseaux profonds (**CNN, RNN, Transformers**) est devenue indispensable pour les données les plus complexes et les plus massives : les **images et vidéos** d'une part, le **langage humain** d'autre part.

Aa

Traitement du langage naturel (NLP)

L'art de faire comprendre, interpréter et générer le langage humain par les machines.

Ex. : traduction, assistants conversationnels, analyse de sentiments

◉

Vision par ordinateur

La science qui permet aux machines de « voir » et de comprendre le contenu visuel du monde (images et vidéos).

Ex. : diagnostic médical, conduite autonome, contrôle qualité

QUIZ · À VOUS DE JOUER

Quel élément du neurone artificiel joue le rôle des synapses ?

- Les entrées x
- **Les poids w**
- La fonction d'activation
- La sortie y

Réponse Les poids w : comme les synapses, ils modulent la force (l'importance) de chaque signal d'entrée.